

Classification under Uncertainty

Orna Kupferman @

The Hebrew University of Jerusalem, Israel

Ofer Leshkowitz @

The Hebrew University of Jerusalem, Israel

Abstract

Consider a fixed number of disjoint regular languages $L_1, \dots, L_k \subseteq \Sigma^*$. A *classifier* for $L_1, \dots, L_k$ is a transducer that receives each moment t in time an input letter $\sigma_t \in \Sigma$ and outputs an index in $\{1, \dots, k\}$ such that if the word $\sigma_1 \dots \sigma_t$, generated so far, is in some (unique) language L_i , then this index is i . Classifiers arise naturally in runtime monitoring and online stream processing, where a system must continuously determine which of several specifications or behaviors is currently being realized. The problem of generating classifiers of minimal size has been well studied.

In many applications, the input alphabet is of the form 2^P , for a finite set P of signals. There, the complexity of classification stems not only from the languages but also from the presence of uncertainty, namely when the valuation to some of the signals may not be known. We introduce and study *classification under uncertainty*, where the input words may be partially observed. We consider three sources for uncertainty: (1) *Given*: the input to the problem specifies which signals may be sensed after each behavior. (2) *Privacy*: the input includes a list of secret behaviors, and the classifier should restrict sensing so that secrets are not revealed. (3) *Budget*: Sensing of signals incurs a cost, which the classifier should minimize.

2012 ACM Subject Classification Theory of computation

Keywords and phrases Formal methods, Automata, Incomplete Information

Digital Object Identifier 10.4230/LIPIcs.CONCUR.2026..

1 Introduction

Already in the 1970s, researchers have studied the problem of *DFA identification* from data: given a set D of words, labeled positively and negatively, we seek a deterministic automaton over finite words (DFA) $\mathcal{A}$ that agrees with D . That is, $\mathcal{A}$ accepts all the positively-labeled words, rejects all the negatively-labeled words, and may accept or reject all other words. A fundamental result by Gold shows that the freedom with respect to unlabelled words makes the problem of finding a minimal identifying DFA NP-complete [12]. The identification problem has numerous heuristics and applications [27, 15]. The massive use of data of rich types motivates two natural generalization of the problem. First, rather than a finite set D of labeled words, one may consider two DFAs that describe the (possibly infinite) sets of positive and negative words. This is called *separation*: Given two DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ with disjoint languages, we seek a DFA $\mathcal{A}$ such that $L(\mathcal{A}_1) \subseteq L(\mathcal{A})$ and $L(\mathcal{A}) \cap L(\mathcal{A}_2) = \emptyset$. NP-hardness of DFA separation can be easily obtained by a reduction from DFA identification, and researchers have studied variants like strict separation [11, 17], regular separators of general languages [5], as well as separation of regular languages by weaker classes of languages, e.g., FO-definable languages [23] or piecewise testable languages [6].

A second generalization allows the words in D to be labeled by more than two labels. Then, rather than a DFA that agrees with D , we seek a *transducer* that agrees with D . Essentially, a transducer is a DFA whose states are labeled by output letters, refining the accept/reject label in DFAs. Combining the two extensions, one gets the *classification* problem: For $k \geq 1$ and k DFAs $\mathcal{A}_1, \dots, \mathcal{A}_k$ with disjoint languages over some alphabet Σ , we seek a transducer that, at each moment t in time, reads an input letter $\sigma_t \in \Sigma$ and outputs an index in $\{1, \dots, k\}$ such that whenever the word $\sigma_1 \dots \sigma_t$ generated so far belongs to $L(\mathcal{A}_i)$, the output index is i . Again, it is easy to see that as long as k is constant, results on identification apply also to the classification problem. Indeed, a classifying transducer can be defined on top of the product of the underlying

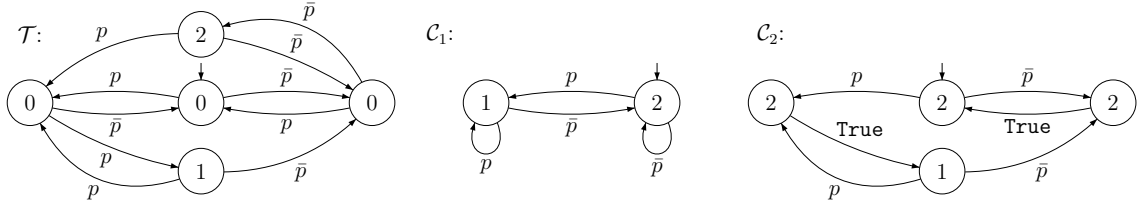


DFAs, a candidate minimal transducer can be checked in polynomial time, implying membership in NP, and NP-hardness follows from the NP-hardness of the separation problem, namely the case $k = 2$. When k is not constant, the product of the underlying DFAs is exponential in k , the problem is PSPACE-complete, and the analysis is less interesting, as the exponent in k dominates other factors. In order to cancel the dominance of k , we assume that the input to the classification problem is a transducer $\mathcal{T}$ that maps each word to an index in $\{0, 1, \dots, k\}$, inducing k disjoint languages – for $i \in \{1, \dots, k\}$, the language L_i consists of words mapped to index i , as well as *don't care* words – these that $\mathcal{T}$ map to 0.

Classifiers are used in runtime monitoring and control of reactive systems, where decisions must be taken online based on a growing execution prefix. Typical applications include fault detection, protocol identification, and intrusion detection, where the system must continuously determine which of several behaviors or specifications is currently being realized. This view is closely related to runtime verification [21, 13], where executions are monitored against formal specifications, and to stream or event processing, where patterns over data streams must be recognized incrementally [4]. Classifiers can be seen as a refinement of monitors that not only detect violations but also distinguish between multiple hypotheses or modes of operation.

In many of the above applications, the input alphabet is of the form 2^P , for a finite set P of signals. This corresponds to the standard representation of computations in formal methods, where behaviors correspond to a sequence of valuations [24, 28]. In these settings, the challenge of classifying stems not only from the languages but also from the need to make decisions in the presence of uncertainty, namely when the valuation to some of the signals may not be known.

► **Example 1.** Consider a setting with $P = \{p\}$, thus behaviors correspond to languages over the alphabet of two valuations, which we denote by $\{p, \bar{p}\}$. Then, **True** denotes an unknown valuation. Consider the transducer $\mathcal{T}$ on the left of Figure 1.



■ **Figure 1** The specification transducer $\mathcal{T}$, and two classifiers that satisfy $\mathcal{T}$

The transducer $\mathcal{T}$ specifies two disjoint languages $L_1, L_2 \subseteq \{p, \bar{p}\}^*$. It does so by leading each word in $\{p, \bar{p}\}^*$ to a state labeled 0 (the word is not in $L_1 \cup L_2$), 1 (the word is in L_1), or 2 (the word is in L_2). It is not hard to see (at least to readers who do not miss the arrow pointing to the initial state...) that $L_1 = (\text{True} \cdot \text{True})^* \cdot p \cdot p$ and $L_2 = (\text{True} \cdot \text{True})^* \cdot \bar{p} \cdot \bar{p}$.

The transducer $\mathcal{C}_1$ is a classifier: it maps each word in $\{p, \bar{p}\}^*$ to either 1 or 2. Specifically, it maps to 1 exactly all words in $\text{True}^* \cdot p$ and maps to 2 all words in $\text{True}^* \cdot \bar{p}$. The classifier $\mathcal{C}_1$ satisfies $\mathcal{T}$. Indeed, all words in L_1 are mapped to 1 and all words in L_2 are mapped to 2. It is not hard to prove that $\mathcal{C}_1$ is a minimal classifier for $\mathcal{T}$. The classifier $\mathcal{C}_1$, however, needs to know the valuation to the signal p in all its transitions.

The transducer $\mathcal{C}_2$ is also a classifier for $\mathcal{T}$. Indeed, $\mathcal{C}_2$ maps to 1 exactly all words in $(\text{True} \cdot \text{True})^* \cdot p \cdot \text{True}$, maps to 2 exactly all words in $(\text{True} \cdot \text{True})^* \cdot \bar{p} \cdot \text{True}$, and so all words in L_1 are mapped to 1 and all words in L_2 are mapped to 2. Essentially, the transducer $\mathcal{C}_2$ uses the fact that the specification transducer $\mathcal{T}$ does not care for the classification of words whose last two letters are different, and uses this fact in order to give up the sensing of p in all even positions. ◀

We introduce and study *classification under uncertainty*, where the input words are only partially observed. That is, rather than reading at each moment a valuation to all the signals in P , the

classifier may sense a valuation to a subset of them – possibly a different subset at each moment – and its transitions depend only on the sensed signals. Our contribution is related to work on monitoring and runtime verification under partial observability, for example by allowing missing events, indistinguishability relations, or assumptions on the hidden part of the trace [20, 3, 9]. This line of work primarily focuses on extending the monitoring pipeline so as to produce sound verdicts despite incomplete observations. For example, [3] proposes to restrict the space of executions by adding assumptions on the missing information, while [9] reasons about sets of traces consistent with the observed one, maintaining a belief set over the hidden behaviors. In contrast, we study the theoretic foundation of the problem: we explicitly model the sources of uncertainty and study their effect on the classification problem, yielding a structural and complexity-theoretic understanding of classification under uncertainty.

We consider three sources for uncertainty. The first is when uncertainty is determined by the environment and is known in advance. More formally, uncertainty is specified by a *sensing constraint* – a transducer $\mathcal{U}$ that maps each word $w \in (2^P)^*$ to the set of signals whose valuation is observed (in particular, by $\mathcal{U}$) after w is read. The goal is to generate, given a language specification $\mathcal{T}$ and a sensing constraint $\mathcal{U}$, a classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$. We show that, surprisingly, while the problem of deciding whether such a classifier exists is NLOGSPACE-complete, a minimal classifier that satisfies $\mathcal{U}$ may be exponential in $\mathcal{T}$. Intuitively, while a negative answer to the decision problem has a short witness, induced by following the runs of $\mathcal{T}$ on two partially-sensed short words, a witness for a positive answer involves all words, and thus has to remember arbitrary subsets of states of $\mathcal{T}$.

The two other sources of uncertainty concern settings in which it is of the classifier’s interest to generate uncertainty, either for ensuring *privacy* or for minimizing *sensing cost*.

Privacy is an important quality measure for reactive systems: we seek systems that allow the underlying components not to reveal information they prefer to keep private. For reactive systems, researchers have suggested to reason about privacy either by hyper-properties [10], or, more directly, by letting the users specify *secrets* whose truth value should be kept unknown [18, 19]. For the problem of classification, we would like the classifier to observe the input word in a way that enables correct classification without revealing secret information.

Consider for example the language specification $\mathcal{T}$ from Example 1. Suppose we want to classify the words according to $\mathcal{T}$, yet we want to keep the value of the secret “there is a position 0 mod 3 in which p holds” unknown to the classifier. Clearly, the secret is not hidden from Classifier $\mathcal{C}_1$, as it full observes all the valuations to p . The secret is not hidden from Classifier $\mathcal{C}_2$ either, as reading the input word $p \cdot p \cdot p$, the Classifier $\mathcal{C}_2$ observes the third letter, and can deduce that the secret holds. In Example 9, we describe a classifier $\mathcal{C}_3$ for $\mathcal{T}$ that does hide the secret.

Formally, the problem of *privacy-preserving classification* gets as input, in addition to the language specification $\mathcal{T}$, also a *secret* $\mathcal{H}$, which is also a language-specification transducer; thus $\mathcal{H}$ maps each word in $(2^P)^*$ to an index in $\{0, \dots, m\}$, for some $m \geq 1$. The goal is to generate a classifier that satisfies $\mathcal{T}$ and respects a sensing restriction $\mathcal{U}$ that hides $\mathcal{H}$: for every word $w \in (2^P)^*$, an observer restricted by $\mathcal{U}$ cannot classify w according to $\mathcal{H}$. We show that a privacy-preserving classification can be solved in EXPTIME, a classifier may be exponential in $\mathcal{T}$, and the problem is PSPACE-hard, already for secrets of size 2. Our results can be extended to multiple secrets and richer types of secrets.

In many applications, sensing the evaluation of signals involves the operation of sensors and thus has a cost. In the third type of uncertainty, we assume that whenever a classifier senses a signal, this operation has a cost, and we seek a classifier that minimizes the expected sensing cost. Expectation is measured with respect to words of increasing lengths under a given distribution of the valuations. Consider for example the classifiers $\mathcal{C}_1$ and $\mathcal{C}_2$ in Example 1, and assume that each sensing of p incurs a cost of 1, and that p is assigned **True** and **False** with probability $\frac{1}{2}$ each. Since $\mathcal{C}_1$ senses p in all states, its expected sensing cost is 1. Since $\mathcal{C}_2$ senses p in exactly all even



positions, its expected sensing cost is only $\frac{1}{2}$.

As Example 1 demonstrates, minimizing the size of classifiers may conflict with minimizing their sensing cost. We show that the tradeoff is unbounded, in fact already for $P = \{p\}$ and $k = 2$. That is, we show that for every $n \geq 1$, there is a transducer $\mathcal{T}_n$ of size $O(n)$ such that the smallest classifier that satisfies $\mathcal{T}_n$ has 2 states, it needs to constantly sense p , and there is also a classifier of size $O(n)$ that satisfies $\mathcal{T}_n$, with sensing cost $\frac{1}{n}$ only. More interestingly, we show that, in general, optimal sensing may be obtained only in a classifier that is exponentially larger than the language specification $\mathcal{T}$. This complexity is carried over to the problem of finding a minimally-sensing transducer, which we solve in EXPTIME and show to be PSPACE-hard.

From a technical point of view, all three sources of uncertainty are handled via a game-theoretic approach to the corresponding problems. Given a language specification, we define a two-player safety game in which the actions of Classifier correspond to choosing a set of signals to be sensed and the actions of Environment correspond to choosing a valuation to the signals. The goal of Classifier is to stay in a safe region in which it can assign labels to all words generated during the interaction despite its partial sensing. Sensing constraints limit the actions of Classifier, privacy limits the safe region, and sensing cost makes the game a stochastic mean-payoff game. In Section 7, we discuss how the different variants influence the complexity of the corresponding problems, and explain the challenges that make some of them still open.

2 Preliminaries

2.1 Automata and Transducers

A *nondeterministic automaton on finite words* (NFA, for short) is $\mathcal{A} = \langle \Sigma, Q, q_0, \delta, \alpha \rangle$, where Q is a finite set of states, $q_0 \in Q$ is an initial state, $\delta : Q \times \Sigma \rightarrow 2^Q$ is a transition function, and $\alpha \subseteq Q$ is a set of accepting states. A run of $\mathcal{A}$ on a word $w = \sigma_1 \cdot \sigma_2 \cdots \sigma_n \in \Sigma^*$ is a sequence of states $q_0, q_1, \dots, q_n$ such that $q_{i+1} \in \delta(q_i, \sigma_{i+1})$ for all $i \geq 0$. A run is accepting if $q_n \in \alpha$. A word $w \in \Sigma^*$ is accepted by $\mathcal{A}$ if there is an accepting run of $\mathcal{A}$ on w . The language of $\mathcal{A}$, denoted $L(\mathcal{A})$, is the set of words that $\mathcal{A}$ accepts. For a state $q \in Q$, we use $\mathcal{A}^q$ to denote $\mathcal{A}$ with initial state q . We sometimes refer to δ as a function $\delta : 2^Q \times \Sigma^* \rightarrow 2^Q$, thus extend it to set of states and to words: For every set $S \subseteq Q$, we have that $\delta(S, \epsilon) = S$; for every letter $\sigma \in \Sigma$, we have that $\delta(S, \sigma) = \bigcup_{q \in S} \{\delta(q, \sigma)\}$; and for every word $w \in \Sigma^*$, we have that $\delta(S, w \cdot \sigma) = \delta(\delta(S, w), \sigma)$. When δ is such that $|\delta(q, \sigma)| = 1$ for all $q \in Q$ and $\sigma \in \Sigma$, then $\mathcal{A}$ is *deterministic* (DFA, for short). We then assume that δ is of the form $\delta : Q \times \Sigma \rightarrow Q$, thus its extension to words is of the form $\delta : Q \times \Sigma^* \rightarrow Q$.

Consider two alphabets Σ_I and Σ_O , referred to as the input and output alphabet, respectively. A (Σ_I/Σ_O) -transducer $\mathcal{T}$ is similar to a deterministic automaton over the alphabet Σ_I , except that instead of an acceptance condition, each state is labelled by a letter in Σ_O . Formally, $\mathcal{T} = \langle \Sigma_I, \Sigma_O, Q, q_0, \delta, \tau \rangle$ has $\delta : Q \times \Sigma_I \rightarrow Q$, and $\tau : Q \rightarrow \Sigma_O$ maps each state to a letter in Σ_O . We extend τ to sets of states, where for each $S \subseteq Q$, we have that $\tau(S) = \bigcup_{q \in S} \{\tau(q)\}$. As with automata, we talk about the run of $\mathcal{T}$ on words in Σ_I^* , and extend δ to words. For a word $w \in \Sigma_I^*$, we use $\mathcal{T}(w)$ to denote $\tau(\delta(q_0, w))$, namely the label of the state that $\mathcal{T}$ reaches when it reads w .

2.2 Classification

Consider an alphabet Σ . A *specification* is a $(\Sigma/\{0, 1, \dots, k\})$ -transducer $\mathcal{T}$, for some $k \geq 2$, which we refer to as the *width* of $\mathcal{T}$. Each specification defines k disjoint languages $L_1, \dots, L_k \subseteq \Sigma^*$, where for all $i \in \{1, \dots, k\}$, we have $L_i = \{w : \mathcal{T}(w) = i\}$. The words that are mapped by $\mathcal{T}$ to 0 may be viewed as ‘don’t care’ words. We say $\mathcal{T}$ *separates* two words $w, w' \in \Sigma^*$, denoted $w \not\approx_{\mathcal{T}} w'$, if $\mathcal{T}$ cares about both w and w' and map them to different languages. That is, $0 \neq \mathcal{T}(w) \neq \mathcal{T}(w') \neq 0$.



As explained in Appendix B.1, describing k disjoint languages by a $(\Sigma/\{0, 1, \dots, k\})$ -transducer rather than by k DFWs leads to a tighter complexity analysis.

A *classifier* is a $(\Sigma/\{1, \dots, k\})$ -transducer $\mathcal{C}$, for some *width* $k \geq 2$. Note that while specifications have don't care words, the classifier $\mathcal{C}$ induces a partition of Σ^* to k disjoint languages $C_1, \dots, C_k \subseteq \Sigma^*$, where for all $i \in \{1, \dots, k\}$, we have $C_i = \{w : \mathcal{C}(w) = i\}$.

We say that a classifier $\mathcal{C}$ *satisfies* a specification $\mathcal{T}$ (or that $\mathcal{C}$ is a classifier for $\mathcal{T}$, for short) if $\mathcal{C}$ and $\mathcal{T}$ have the same width, and for every word $w \in \Sigma^*$ and $i \in \{1, \dots, k\}$, if $\mathcal{T}(w) = i$, then $\mathcal{C}(w) = i$. That is, words that $\mathcal{T}$ don't care about may be mapped arbitrarily by $\mathcal{C}$, and other words should keep their mapping by $\mathcal{T}$.

It is not hard to see that a violation of classification can be witnessed by a short word, leading to easy model checking for classifiers (see proof in Appendix C.1).

► **Theorem 2.** *Given a specification $\mathcal{T}$ and a classifier $\mathcal{C}$, the problem of checking whether $\mathcal{C}$ satisfies $\mathcal{T}$ is NLOGSPACE-complete.*

2.3 Sensing

We study languages over an alphabet $\Sigma = 2^P$, for a finite set P of signals. A letter $\sigma \in \Sigma$ corresponds to a truth assignment to the signals. When we define languages over Σ , we use predicates on P in order to denote sets of letters. For example, if $P = \{a, b\}$, then the expression $(\text{True})^* \cdot a \cdot b \cdot (\text{True})^*$ describes all words over 2^P that contain a subword in $\{\{a\}, \{a, b\}\} \cdot \{\{b\}, \{a, b\}\}$. Let Ψ_P denote the set of predicates over P . Describing transducers over input alphabet 2^P , we use predicates in Ψ_P in order to describe the transitions symbolically. Thus, transitions are described by a partial function $\delta : Q \times \Psi_P \rightarrow Q$, where for every state $q \in Q$ and letter $\sigma \in 2^P$, there is exactly one predicate $\theta \in \Psi_P$ such that σ satisfies θ and $\delta(q, \theta)$ is defined. The run of $\mathcal{T}$ on a word $w = \sigma_1 \cdot \sigma_2 \cdots \sigma_m \in \Sigma^*$ is then the sequence of states $q_0, q_1, \dots, q_m$ such that $q_{i+1} = \delta(q_i, \theta_{i+1})$, where for all $i \geq 0$, the predicate θ_{i+1} satisfies σ_{i+1} .

Consider a $(2^P/\Sigma_O)$ -transducer $\mathcal{T} = \langle 2^P, \Sigma_O, Q, q_0, \delta, \tau \rangle$. For a state $q \in Q$ and a signal $p \in P$, we say that p is *sensed in* q if, intuitively, the value of p affects the destination of at least one transition from q . Formally, there exists an assignment $\sigma \in 2^P$ such that $\delta(q, \sigma \setminus \{p\}) \neq \delta(q, \sigma \cup \{p\})$. We use $\text{sensed}(q)$ to denote the set of signals sensed in q .

For a set $P' \subseteq P$, let $\sim_{P'} \subseteq 2^P \times 2^P$ be the equivalence relation on 2^P defined by $\sigma \sim_{P'} \sigma'$ iff $\sigma \cap P' = \sigma' \cap P'$. That is, σ and σ' are indistinguishable when one observes only the signals in P' . We say that two words $w \in (2^P)^*$ and $w' \in (2^P)^*$ are *indistinguishable* by $\mathcal{T}$, denoted $w \sim_{\mathcal{T}} w'$, if $w = \sigma_1 \cdots \sigma_m$ and $w' = \sigma'_1 \cdots \sigma'_m$ have the same length, and for all $i \geq 1$, we have that $\sigma_i \sim_{\text{sensed}(q_i)} \sigma'_i$, where $q_i = \delta(q_0, \sigma_1 \cdots \sigma_{i-1}) = \delta(q_0, \sigma'_1 \cdots \sigma'_{i-1})$. That is, w and w' are indistinguishable if for every position, the two letters in that position are indistinguishable with respect to the signals sensed in the state reached by $\mathcal{T}$ after reading their prefix up to that position. Note that indistinguishability implies that w and w' induce the same run in $\mathcal{T}$, and so $\mathcal{T}(w) = \mathcal{T}(w')$.

Consider a set P of signals and output alphabet Σ_O . We say that a $(2^P/\Sigma_O)$ -transducer $\mathcal{T} = \langle 2^P, \Sigma_O, Q, q_0, \delta, \tau \rangle$ *respects* a $(2^P/2^P)$ -transducer $\mathcal{U} = \langle 2^P, 2^P, R, r_0, \delta', \tau' \rangle$ if for every word $w \in (2^P)^*$, the set of signals sensed in the state $\delta(q_0, w)$ is contained in the set of signals $\mathcal{U}(w)$. A *sensing constraint* is a $(2^P/2^P)$ -transducer $\mathcal{U}$ that respects itself. That is, for every word $w \in (2^P)^*$, we have that $\text{sensed}(\delta'(r_0, w)) \subseteq \mathcal{U}(w)$. Intuitively, a sensing constraint $\mathcal{U}$ specifies the set of signals that can be sensed after reading each word, and the fact that $\mathcal{U}$ respects itself means that a transducer that respects it can follow its constraints (indeed, such a transducer can execute $\mathcal{U}$ despite its partial sensing).

The *sensing cost* of a state $q \in Q$, denoted $\text{sense}(q)$, is the number of signals sensed in q , thus $\text{sense}(q) = |\text{sensed}(q)|$. For a finite run $r = q_0, \dots, q_m$ of $\mathcal{T}$, the sensing cost of r , denoted $\text{sense}(r)$, is the average number of sensors that $\mathcal{T}$ uses during r , thus $\text{sense}(r) = \frac{1}{m} \sum_{i=0}^m \text{sense}(q_i)$. For a



word $w \in (2^P)^*$, the sensing cost of w , denoted $sense(w)$, is the sensing cost of the run of $\mathcal{T}$ on w . Finally, the sensing cost of $\mathcal{T}$, denoted $sense(\mathcal{T})$, is the expected sensing cost of words of length that tends to infinity, where we assume that the letters in 2^P are uniformly distributed. That is, in every point in time, each signal in P is assigned **True** with probability $\frac{1}{2}$ and is assigned **False** with probability $\frac{1}{2}$ independently. As detailed below, the value $sense(\mathcal{T})$ can be found by analyzing the *limiting distribution* of a random walk on the Markov chain induced by $\mathcal{T}$.

We first need some definitions. A *Markov chain* $M = \langle S, s_0, P \rangle$ consists of a finite state space S , an initial state s_0 , and a stochastic transition matrix $P : S \times S \rightarrow [0, 1]$. That is, for all $s \in S$, we have $\sum_{s' \in S} P(s, s') = 1$. An infinite random walk on M starts in s_0 and repeatedly move between states according to P . Recall that we assume that signals are assigned **True** and **False** with probability $\frac{1}{2}$. Accordingly, a transducer $\mathcal{T} = \langle 2^P, \Sigma_O, Q, q_0, \delta, \tau \rangle$ induces a Markov chain $M_{\mathcal{T}} = \langle Q, q_0, P \rangle$, where for all states $q, q' \in Q$, we have $P(q, q') = \frac{1}{2^{|P|}} |\{\sigma \in 2^P : \delta(q, \sigma) = q'\}|$.

In Appendix A, we detail how each Markov chain $M = \langle S, s_0, P \rangle$ has a *limiting distribution* $\pi : S \rightarrow [0, 1]$, mapping each state s to the expected proportion of time that a random walk on M that starts in s_0 and proceeds according to P spends in s . By [14], the limiting distribution can be computed in polynomial time by solving a system of linear equations. Then, by [1], the sensing cost of a transducer $\mathcal{T}$ can be calculated by examining the sensing cost of its states and their limiting distributions in $M_{\mathcal{T}}$. Specifically, if the limiting distribution of $M_{\mathcal{T}}$ is $\pi : Q \rightarrow [0, 1]$, then, $sense(\mathcal{T}) = \sum_{q \in Q} \pi(q) \cdot sense(q)$.

For example, the classifier $\mathcal{C}_2$ from Example 1 has $\pi(q) = \frac{1}{4}$ for all states q . Since two states sense p and two states do not sense p , we have that $sense(\mathcal{T}) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$.

► **Remark 3.** It is easy to extend the setting to one in which the assignments to the signals are not uniform or fixed. Rather, they are generated by a Markov chain M_{assign} , each of its states describe the probability of each signal to be assigned **True**. Indeed, in this case we define $M_{\mathcal{T}}$ by taking the product of $\mathcal{T}$ with M_{assign} , and all our results apply. Likely, it is easy to let different signals to have different costs, updating the definition of $sense(q)$ to $\sum_{p \in sensed(q)} sense(p)$. ◀

3 The Classification Game

In this section we describe the game-theoretic approach to sensing under uncertainty. Given a specification $\mathcal{T}$, we describe a game between Classifier and Environment, such that winning strategies for Classifier correspond to different sensing constraints.

A *2-player safety game* between Classifier and Environment is $\mathcal{G} = \langle V_C, V_E, v_0, v_{\perp}, E \rangle$, where V_C and V_E are disjoint sets of vertices, owned by Classifier and Environment, respectively, $v_0 \in V_C$ is an initial vertex, $v_{\perp}$ is a losing vertex, and $E \subseteq (V_C \times V_E) \cup (V_E \times (V_C \cup \{v_{\perp}\}))$ is a total edge relation. In the beginning of a play in the game, a token is placed on v_0 . Then, in each turn, the player that owns the vertex that hosts the token chooses a successor vertex and moves the token to it. Together, the players generate a *play* $\rho = v_0, v_1, \dots$ in G , namely an infinite path that starts in v_0 and respects E : for all $i \geq 0$, we have that $(v_i, v_{i+1}) \in E$. If the play reaches $v_{\perp}$, it ends, and Environment wins. If the play continues forever, Classifier wins.

A (*positional*) *strategy* for Classifier is a function $f_C : V_C \rightarrow V_E$ that maps each vertex owned by Classifier to a successor to which Classifier moves the token, in a way that respects E . That is, for every $v \in V_C$, we have that $(v, f_C(v)) \in E$. A strategy for Environment is defined similarly, as $f_E : V_E \rightarrow V_C \cup \{v_{\perp}\}$. A *profile* is a tuple $\pi = \langle f_C, f_E \rangle$ of strategies, one for each player. The *outcome* of a profile $\pi = \langle f_C, f_E \rangle$ is the play obtained when the players follow their strategies in π . Formally, $\text{Outcome}(\pi) = v_0, v_1, v_2, \dots$ is such that for all $j \geq 0$, either $v_j = v_{\perp}$, or $v_j \in V_C$, in which case $v_{j+1} = f_C(v_0, v_1, \dots, v_j)$, or $v_j \in V_E$, in which case $v_{j+1} = f_E(v_0, v_1, \dots, v_j)$. A strategy f_C is *winning* for Classifier if for all strategies f_E of Environment, we have that $\text{Outcome}(\langle f_C, f_E \rangle)$ is winning for Classifier.



Consider a specification $\mathcal{T} = \langle 2^P, \{0, \dots, k\}, Q, q_0, \delta, \tau \rangle$. Let $\text{legit}(\mathcal{T}) \subseteq 2^Q$ be the set of subsets of Q that do not contain states labeled by different labels in $\{1, \dots, k\}$. Formally, $S \in \text{legit}(\mathcal{T})$ iff there is $i \in \{1, \dots, k\}$ such that for all $q \in S$, we have that $\tau(q) \in \{0, i\}$. Intuitively, $\text{legit}(\mathcal{T})$ includes subsets with respect to which uncertainty is legitimate, in the sense that a classifier that does not distinguish between the states in S can still classify all of them to the same language.

We define $\mathcal{G}_{\mathcal{T}}$ as follows. Intuitively, in each round in the game, Classifier decides which signals in P it senses, and Environment generates a letter in (2^P) . Together, the players generate a word $w \in (2^P)^\omega$ – the one obtained by concatenating the letters generated by Environment, along with the partial view of Classifier on w . Formally, $\mathcal{G}_{\mathcal{T}} = \langle V_C, V_E, v_0, v_\perp, E \rangle$, where $V_C = \text{legit}(\mathcal{T})$, $V_E = \text{legit}(\mathcal{T}) \times 2^P$, $v_0 = \{q_0\}$, and E is defined as follows. For a state $s \in Q$, a set $P' \subseteq P$ of sensed signals, and an assignment $\sigma \in 2^P$, we define $\text{succ}(s, P', \sigma)$ as the set of possible successors of s when the classifier senses only signals in P' and the environment assigns σ . Formally, $\text{succ}(s, P', \sigma) = \{\delta(s, \sigma') : \sigma' \in 2^P, \sigma \sim_{P'} \sigma'\}$. Then, for a set $S \subseteq Q$ of states, we extend this to $\text{succ}(S, P', \sigma) = \bigcup_{s \in S} \text{succ}(s, P', \sigma)$. This represents the set of states reachable from S given that the classifier observes only signals in P' and the environment chooses assignment σ .

- For every $S \in \text{legit}(\mathcal{T})$ and $P' \subseteq P$, we include the edge $\langle S, \langle S, P' \rangle \rangle \in E$. Thus, from a vertex in V_C , Classifier chooses the set P' of signals to sense in the current legitimate set S .
 - For a state $\langle S, P' \rangle \in V_E$ and a letter $\sigma \in 2^P$, Environment has a move from $\langle S, P' \rangle$ associated with σ , and the next vertex is determined as follows.
 - If $\text{succ}(S, P', \sigma) \in \text{legit}(\mathcal{T})$, then $\langle \langle S, P' \rangle, \text{succ}(S, P', \sigma) \rangle \in E$.
 - Otherwise, $\langle \langle S, P' \rangle, v_\perp \rangle \in E$.
- In other words, if the successors of the states in S under P' and σ form a legitimate set, the game continues there; otherwise, it moves to $v_\perp$.

Every winning positional strategy f for Classifier induces the sensing constraint $\mathcal{U}_f = \langle 2^P, 2^P, \text{legit}(\mathcal{T}), \{q_0\}, \delta', \tau' \rangle$, where for all $S \in \text{legit}(\mathcal{T})$ with $f(S) = \langle S, P' \rangle$, we have that $\tau'(S) = P'$, and for all $\sigma \in 2^P$, we have that $\delta'(S, \sigma) = \text{succ}(S, P', \sigma)$. The fact f is winning implies that it also induces a classifier $\mathcal{C}_f$ that satisfies $\mathcal{T}$ and respects the sensing constraint $\mathcal{U}_f$. Formally, $\mathcal{C}_f = \langle 2^P, 2^P, \text{legit}(\mathcal{T}), \{q_0\}, \delta_f, \tau' \rangle$, where for all $S \in \text{legit}(\mathcal{T})$ with $f(S) = \langle S, P' \rangle$ and $\sigma \in 2^P$, we have that $\delta_f(S, \sigma) = \text{succ}(S, P', \sigma)$, and $\tau'(S) = \max\{\max \gamma(S), 1\}$. Note that since $S \in \text{legit}(\mathcal{T})$, then $\tau(S) \subseteq \{0, i\}$, for some $i \in \{1, \dots, k\}$. Hence, the definition of τ' is such that $\tau'(S)$ is (arbitrarily set to) 1 if $\tau(S) = \{0\}$, and is i otherwise. By the definitions, we have the following.

► **Lemma 4.** *Consider a positional winning strategy f for Classifier in $\mathcal{G}_{\mathcal{T}}$. The classifier $\mathcal{C}_f$ satisfies $\mathcal{T}$ and respects the sensing constraint $\mathcal{U}_f$.*

4 Classification with Constrained Sensing

The problem of classification with constrained sensing gets as input a language specification $\mathcal{T}$ and a sensing constraint $\mathcal{U}$, both over the same alphabet 2^P , and has to return a classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$, or determine that no such classifier exists.

In order to solve the problem, we first augment the states of $\mathcal{T}$ by states of $\mathcal{U}$, indicating the set of signals that may be sensed. Let $\mathcal{T} = \langle 2^P, \{0, \dots, k\}, Q, q_0, \delta, \tau \rangle$, and consider a sensing constraint $\mathcal{U} = \langle 2^P, 2^P, R, r_0, \delta', \tau' \rangle$. We define the specification $\mathcal{T}^{\mathcal{U}} = \langle 2^P, \{0, \dots, k\}, Q \times R, \langle q_0, r_0 \rangle, \delta^{\mathcal{U}}, \tau^{\mathcal{U}} \rangle$, where for every state $\langle q, r \rangle \in Q \times R$ and letter $\sigma \in 2^P$, we have that $\delta^{\mathcal{U}}(\langle q, r \rangle, \sigma) = \langle \delta(q, \sigma), \delta'(r, \sigma) \rangle$. Also, $\tau^{\mathcal{U}}(\langle q, r \rangle) = \tau(q)$. It is easy to see that since $\mathcal{U}$ is a deterministic transducer with transitions defined for all letters in 2^P , the specifications $\mathcal{T}$ and $\mathcal{T}^{\mathcal{U}}$ are equivalent, in the sense that for all $w \in (2^P)^*$, we have that $\mathcal{T}(w) = \mathcal{T}^{\mathcal{U}}(w)$.

Consider the game $\mathcal{G}_{\mathcal{T}^{\mathcal{U}}} = \langle V_C, V_E, v_0, v_\perp, E \rangle$, namely the game defined in Section 3 for the specification $\mathcal{T}^{\mathcal{U}}$. Recall that $V_C = \text{legit}(\mathcal{T}^{\mathcal{U}})$, and so each state in V_C is a subset $S \subseteq Q \times R$. We



say that S is *good* if there is $r \in R$ such that $S \subseteq Q \times \{r\}$. That is, all the states in S agree on their state in $\mathcal{U}$. We call r the *witness* of S .

Let $f_{\mathcal{U}} : V_C \rightarrow V_E$ be a strategy for Classifier that follows $\mathcal{U}$ in all good positions. That is, if S is good with witness r , then $f_{\mathcal{U}}(S) = \langle S, \tau'(r) \rangle$. By definition, the vertex $v_0 = \{\langle q_0, r_0 \rangle\}$ is good, with witness r_0 . Moreover, following $f_{\mathcal{U}}$ is the only way for Classifier to win with a strategy that respects $\mathcal{U}$, and it is guaranteed that all vertices visited along a play in $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ are good. As Classifier win a safety game iff it has a positional winning strategy [8], we have the following:

► **Theorem 5.** *Consider a specification $\mathcal{T}$ and a sensing constraint $\mathcal{U}$. Let $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ be the game between Classifier and Environment that correspond to the specification $\mathcal{T}^{\mathcal{U}}$. There is a classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$ iff the strategy $f_{\mathcal{U}}$ is winning in $\mathcal{G}_{\mathcal{T}\mathcal{U}}$.*

Theorem 5 implies that the problem of deciding whether there is a classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$ can be solved by reasoning about $\mathcal{T}^{\mathcal{U}}$. As good and surprising news, we show that even though the state space of $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ is exponential in $\mathcal{T}$, one need not traverse the whole state space of $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ and the problem is NLOGSPACE-complete.

► **Lemma 6.** *Consider a specification $\mathcal{T}$ and a sensing constraint $\mathcal{U}$ with state spaces Q and R , respectively. The following are equivalent.*

- (1) *There is no classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$.*
- (2) *There are words $w, w' \in (2^P)^*$ such that $w \sim_P w'$ and $\mathcal{T}(w) \not\approx_{\mathcal{T}} \mathcal{T}(w')$.*
- (3) *There are words $w, w' \in (2^P)^*$ such that $|w| = |w'| \leq |Q|^2 \cdot |R|$, $w \sim_P w'$, and $\mathcal{T}(w) \not\approx_{\mathcal{T}} \mathcal{T}(w')$.*

Proof. The implications (1) $\rightarrow$ (2) and (3) $\rightarrow$ (1) follow easily from the fact that (1) holds iff the strategy $f_{\mathcal{U}}$ is losing in $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ (see details in Appendix C.2).

We prove here that (2) $\rightarrow$ (3). Consider words $w, w' \in (2^P)^*$ such that $w \sim_{\mathcal{U}} w'$, and $\mathcal{T}(w) \not\approx_{\mathcal{T}} \mathcal{T}(w')$. Since $w \sim_{\mathcal{U}} w'$, the runs of $\mathcal{U}$ on w and w' coincides. Let $r_0, r_1, \dots, r_m$ be this run and let $q_0, q_1, \dots, q_m$ and $q'_0, q'_1, \dots, q'_m$ be the runs of $\mathcal{T}$ on w and w' , respectively. Assume that $m > |Q|^2 \cdot |R|$. Then, there are $0 \leq i_1 < i_2 \leq m$ such that $\langle r_{i_1}, q_{i_1}, q'_{i_1} \rangle = \langle r_{i_2}, q_{i_2}, q'_{i_2} \rangle$. Thus, we can shorten w and w' by removing the subword between positions $i_1 + 1$ and i_2 , and obtain shorter words that still satisfy the same properties. Repeating this process, we can obtain words $x, x' \in (2^P)^*$ of length at most $|Q|^2 \cdot |R|$ such that $x \sim_{\mathcal{U}} x'$ and $\mathcal{T}(x) \not\approx_{\mathcal{T}} \mathcal{T}(x')$. ◀

► **Theorem 7.** *Consider a specification $\mathcal{T}$ and a sensing constraint $\mathcal{U}$. The problem of deciding whether there is a classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$ is NLOGSPACE-complete.*

Proof. Let $\mathcal{T} = \langle 2^P, \{0, \dots, k\}, Q, q_0, \delta, \tau \rangle$ and $\mathcal{U} = \langle 2^P, 2^P, R, r_0, \delta', \tau' \rangle$. We prove that the complementing problem is NLOGSPACE. By Lemma 6, there is no classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$ iff there are words $w, w' \in (2^P)^*$ such that $w, w' \in (2^P)^*$ such that $|w| = |w'| \leq |Q|^2 \cdot |R|$ and $w \sim_P w'$ and $\mathcal{T}(w) \not\approx_{\mathcal{T}} \mathcal{T}(w')$. A nondeterministic logarithmic-space Turing machine can guess such words w and w' letter by letter, maintaining in each iteration the states $r \in R$, $q \in Q$, and $q' \in Q$, of the runs of $\mathcal{U}$ and $\mathcal{T}$ on w and w' , and a counter of the length of the generated words. Note that the constraint $w \sim_{\mathcal{U}} w'$ is fulfilled by guessing the next letters $\sigma_i \in 2^P$ and $\sigma'_i \in 2^P$, of w and w' respectively, so that $\sigma_i \sim_{\tau'(r_{i-1})} \sigma'_i$. The machine accepts if at some point the generated word satisfy the conditions $\tau(q) \neq \tau(q')$ and $\tau(q), \tau(q') \neq 0$, and rejects when the counter exceeds $|Q|^2 \cdot |R|$.

Hardness in NLOGSPACE is by an easy reduction from non-reachability in a directed graph: Given a directed graph $G = \langle V, E \rangle$ and two vertices $s, t \in V$, let 2^P encode the vertices in V , and for every vertex $v \in V$, let $\sigma_v \in 2^P$ be the letter that encodes v . Now, let $\mathcal{T}$ map to 1 all words in $\sigma_s \cdot \text{True}^* \cdot \sigma_t$ that correspond to paths in G and map to 2 all words in $\sigma_s \cdot (2^P \setminus \{\sigma_t\})^*$. Then, let $\mathcal{U}$ enable no sensing. It is easy to see that there is a classifier for $\mathcal{T}$ that respects $\mathcal{U}$ iff there is no path from s to t in G . Indeed, such a classifier may map all words to 2. Also, both $\mathcal{T}$ and $\mathcal{U}$ can be generated in logarithmic space from G . ◀



370 The low complexity of the problem is especially surprising given that a minimal classifier that
 371 classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$ may be exponential in $\mathcal{T}$, as we state below.

372 ► **Theorem 8.** *For every specification $\mathcal{T}$ and sensing constraint $\mathcal{U}$, if there is a classifier that*
 373 *satisfies $\mathcal{T}$ and respects $\mathcal{U}$, there there is also one of size exponential in $\mathcal{T}$ and polynomial in $\mathcal{U}$. This*
 374 *bound is tight: For every $n \geq 1$ there is a set of signals P_n of size $O(n)$, a $(2^{P_n}/\{0, 1, 2\})$ -transducer*
 375 *$\mathcal{T}_n$ with $O(n)$ states, and a sensing constraints $\mathcal{U}_n$ with 2 states, such that every classifier that*
 376 *satisfies $\mathcal{T}_n$ and respects $\mathcal{U}_n$ has at least 2^n states.*

377 **Proof.** By Lemma 4, a classifier that satisfies $\mathcal{T}$ and respects $\mathcal{U}$ can be defined on top of the state
 378 space of $\mathcal{G}_{\mathcal{T}\mathcal{U}}$, restricted to good sets in $\text{legit}(\mathcal{T}, \mathcal{U})$. Thus, the upper bound follows from the bound
 379 on the number of such sets.

380 We continue to the lower bound. For $n \geq 0$, let $[n] = \{0, 1, \dots, n-1\}$. We define $\mathcal{T}_n =$
 381 $\langle 2^{P_n}, Q_n, q_0, \delta_n, \tau \rangle$ as follows.

- 382 ■ $P_n = \{s_0, \dots, s_{n-1}, d_1, \dots, d_{n-1}\}$ consists of n *s-signals* and $n-1$ *d-signals*. We interpret
 383 a valuation $v \in 2^{P_n}$ in two ways. The *odd interpretation* to v , denoted $\text{odd}(v)$, is the pair
 384 $\langle i, j \rangle \in [n] \times \{0, 1\}$ defined as follows. Let $i = \sum_{\ell=1}^{n-1} v(d_\ell)$ be the number of *d-signals* assigned
 385 true in v . Then, $\text{odd}(v) = \langle i, v(s_i) \rangle$. The *even interpretation* of v , denoted $\text{even}(v)$, is the pair
 386 $\langle i, j \rangle \in [n] \times \{0, 1\}$ defined as follows. Let $k = \sum_{\ell=0}^{n-1} v(s_\ell) + \sum_{\ell=1}^{n-1} v(d_\ell)$ be the total number of
 387 *s-* and *d-signals* assigned true in v . Let $i = \lfloor k/2 \rfloor$ and $j = k \pmod{2}$. Then, $\text{even}(v) = \langle i, j \rangle$.
 388 Note that the even and odd interpretations both define surjective mappings $2^{P_n} \rightarrow [n] \times \{0, 1\}$.
 389 Given two interpretations $\langle i, j \rangle, \langle i', j' \rangle \in [n] \times \{0, 1\}$, we say that $\langle i, j \rangle$ and $\langle i', j' \rangle$ are a *couple*,
 390 denoted $\langle i, j \rangle \approx \langle i', j' \rangle$, iff $i = i'$ and $j \neq j'$. Note that $[n] \times \{0, 1\}$ is partitioned into n couples.
 391 Also, for two valuations $v, u \in 2^{P_n}$, we have $\text{odd}(v) \approx \text{odd}(u)$ iff the number $i \in [n]$ of true
 392 *d-signals* is the same for both valuations, and yet $v(s_i) \neq u(s_i)$. In particular, for every $v \in 2^{P_n}$
 393 there exists a valuation $u \in 2^{P_n}$ that is different from v only in the value of one *s-signal*, such
 394 that $\text{odd}(v) \approx \text{odd}(u)$. For the even interpretation, we have $\text{even}(v) \approx \text{even}(u)$ iff the numbers
 395 k_v and k_u of true *s-* and *d-signals* in v and u respectively, satisfy $\{k_v, k_u\} = \{2i, 2i+1\}$ for
 396 some $i \in [n]$. In particular, for every $v \in 2^{P_n}$, there exists a valuation $u \in 2^{P_n}$ that is different
 397 from v only in the value of one signal, such that $\text{even}(v) \approx \text{even}(u)$.
- 398 ■ The state space $Q_n = \{q_0, q_1, q_2\} \cup [n] \times \{0, 1\}$ is partitioned into the set $\{q_0, q_1, q_2\}$ of *odd*
 399 *states*, and the set $\{\langle i, j \rangle : i \in [n], j \in \{0, 1\}\}$ of *even states*.
- 400 ■ The transition function δ_n is defined as follows for every state $q \in Q_n$ and letter $v \in 2^{P_n}$:

$$401 \quad \delta_n(q, v) = \begin{cases} \text{odd}(v) & \text{if } q \in \{q_0, q_1, q_2\} \\ q_1 & \text{if } q = \text{even}(v) \\ q_2 & \text{if } q \approx \text{even}(v) \\ q_0 & \text{otherwise} \end{cases}$$

402

403 It is not hard to see that $\mathcal{T}_n$ alternates between even and odd states, and so when reading a
 404 word $w = v_1 \dots v_m$, the transition on v_i is defined with respect to the odd interpretation of v_i
 405 if i is odd, and with respect to the even interpretation of v_i if i is even. Since the transition
 406 function is defined with respect to the even or odd interpretation of valuations, we sometimes
 407 write $\delta_n(q, \langle i, j \rangle)$ to denote the transition from a state q on a letter v such that $\text{even}(v) = \langle i, j \rangle$
 408 or $\text{odd}(v) = \langle i, j \rangle$, depending on whether q is an even or odd state. Similarly, we refer to letters
 409 in 2^{P_n} as elements of $[n] \times \{0, 1\}$ (where the interpretation is the even or odd interpretation,
 410 depending on the context).

- 411 ■ The labeling function τ is such that $\tau(q_1) = 1, \tau(q_2) = 2$, and for all other states $s \in Q_n$, we
 412 have $\tau(s) = 0$.



413 Consider a word $w \in (2^{P_n})^*$ of even length. The alternation between even and odd states
 414 implies that $\delta_n(q_0, w) = q \in \{q_0, q_1, q_2\}$. By the definition of δ_n , we have that for every $v_1, v_2 \in 2^{P_n}$

$$415 \quad \delta_n(q_0, w \cdot v_1 \cdot v_2) = \delta_n(q, v_1 \cdot v_2) = \delta_n(\text{odd}(v_1), v_2) = \begin{cases} q_1 & \text{if } \text{odd}(v_1) = \text{even}(v_2) \\ q_2 & \text{if } \text{odd}(v_1) \approx \text{even}(v_2) \\ q_0 & \text{otherwise.} \end{cases}$$

416 Accordingly, the two languages L_1^n and L_2^n defined by $\mathcal{T}_n$ are

417 $\blacksquare L_1^n = \{(\text{True} \cdot \text{True})^* \cdot v_1 \cdot v_2 : \text{odd}(v_1) = \text{even}(v_2)\}.$

418 $\blacksquare L_2^n = \{(\text{True} \cdot \text{True})^* \cdot v_1 \cdot v_2 : \text{odd}(v_1) \approx \text{even}(v_2)\}.$

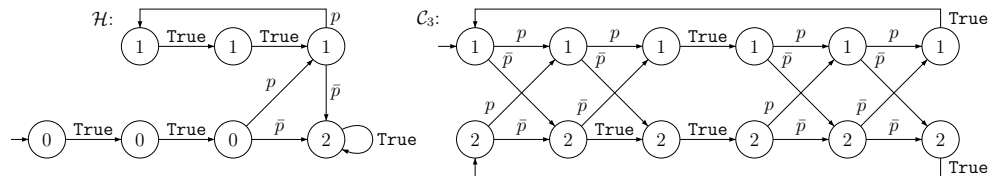
419 That is, L_1^n includes all words of the form $w \cdot v_1 \cdot v_2$ where w is a word of an even length, and
 420 the odd interpretation of v_1 is the same as the even interpretation of v_2 . Likewise, L_2^n includes all
 421 words of the form $w \cdot v_1 \cdot v_2$ where w is a word of an even length, and the odd interpretation of v_1
 422 is a couple of the even interpretation of v_2 .

423 Consider the 2-state sensing constraint $\mathcal{U}_n$ that disables sensing all d -signals at odd positions,
 424 and allows sensing all other signals at all times. Formally, $\mathcal{U}_n = \langle 2^{P_n}, 2^{P_n}, \{r_0, r_1\}, r_0, \delta'_n, \tau'_n \rangle$, where
 425 $\delta'_n(r_0, \sigma) = r_1$ and $\delta'_n(r_1, \sigma) = r_0$ for all $\sigma \in 2^{P_n}$, $\tau'_n(r_0) = \{s_0, \dots, s_{n-1}\}$, and $\tau'_n(r_1) = P_n$. In
 426 Appendix C.3, we prove that there exists a classifier $\mathcal{C}_n^*$ that satisfies $\mathcal{T}_n$ and respects $\mathcal{U}_n$, yet every
 427 such classifier has at least 2^n states. Intuitively, the sensing constraint $\mathcal{U}_n$ forces a correct classifier
 428 for $\mathcal{T}_n$ to maintain in its memory the last assignment to the sensed s -signals after every odd position.
 429 Indeed, for every two assignments S_1 and S_2 to the s -signals, there are two assignments $v_1 \in 2^{P_n}$
 430 and $v_2 \in 2^{P_n}$ that extend S_1 and S_2 , respectively, and there exists a letter $u \in 2^{P_n}$ such that
 431 $\text{odd}(v_1) = \text{even}(u)$ and $\text{odd}(v_2) \approx \text{even}(u)$. Therefore, a correct classifier must move to different
 432 states upon reading v_1 and v_2 from every odd position. Since there are 2^n different assignments to
 433 the s -signals, this implies that there are at least 2^n states in the classifier. ◀

434 5 Classification with Privacy

435 The input to the problem of *classification with privacy* includes, in addition to the language
 436 specification, also a *secret*, given by a $(2^P/\{0, 1, \dots, m\})$ -transducer $\mathcal{H}$. We say that a sensing
 437 constraint $\mathcal{U}$ *hides* $\mathcal{H}$ if for every word $w \in (2^P)^*$ such that $\mathcal{H}(w) \neq 0$, there exists a word $w' \in (2^P)^*$
 438 such that $w \sim_{\mathcal{U}} w'$ and $\mathcal{H}(w) \neq \mathcal{H}(w')$. That is, when a classifier respects $\mathcal{U}$, it cannot deduce the
 439 labeling that $\mathcal{H}$ gives to input words it cares about. We say that a classifier $\mathcal{C}$ *hides* $\mathcal{H}$ if it respects
 440 a sensing constraint that hides $\mathcal{H}$. Our goal is to find, given a language specification $\mathcal{T}$ and a secret
 441 $\mathcal{H}$, a classifier that satisfies $\mathcal{T}$ and hides $\mathcal{H}$.

442 ► **Example 9.** Recall the specification $\mathcal{T}$ from Example 1, and consider the secret $\mathcal{H}$ in Figure 2.
 443 Note that for all words $w = \sigma_1 \cdot \sigma_2 \cdots \sigma_n \in \{p, \bar{p}\}^*$, we have that $\mathcal{H}(w) = 1$ iff $|w| \geq 3$ and $\sigma_i = p$
 444 for all $i = 0 \bmod 3$. As discussed in Section 1, the classifiers $\mathcal{C}_1$ and $\mathcal{C}_2$ do not hide $\mathcal{H}$. Consider the
 445 classifier $\mathcal{C}_3$, also in Figure 2.



■ **Figure 2** The secret $\mathcal{H}$ and the classifier $\mathcal{C}_3$ that satisfies $\mathcal{T}$ and hides $\mathcal{H}$.

446 Note that $\mathcal{C}_3$ is a correct classifier for $\mathcal{T}$. Indeed, $\mathcal{C}_3$ makes use of the fact that it is possible
 447 to satisfy $\mathcal{T}$ while reading only one letter in each suffix $\sigma_{2i-1} \cdot \sigma_{2i}$, for all $i \geq 1$. Indeed, if one
 448 of these letters is p , then $\sigma_1 \cdots \sigma_{2i}$ can be mapped to 1, and if one of them is $\bar{p}$, then it can be
 449 mapped to 2. Since for all $i \geq 1$, either $2i - 1$ or $2i$ is not $0 \bmod 3$, it is thus possible to satisfy $\mathcal{T}$
 450 without revealing $\mathcal{H}$. Formally, $\mathcal{C}_3$ respects a sensing constraint $\mathcal{U}$ that senses σ_i iff $i \neq 0 \bmod 3$.
 451 Hence, every word w such that $|w| \geq 3$ (that is, words that $\mathcal{H}$ cares about) has a word w' such that
 452 $w \sim_{\mathcal{U}} w'$ and $\mathcal{H}(w) \neq \mathcal{H}(w')$. $\blacktriangleleft$

453 **► Theorem 10.** *Classifier with privacy can be solved in EXPTIME.*

454 **Proof.** Given a language specification $\mathcal{T}$ and a secret $\mathcal{H}$, we reduce classification with privacy to
 455 the game defined in Section 3, except that now the definition of legitimate sets reflects also the
 456 need to hide the secret.

457 For simplicity, we assume that $\mathcal{T}$ and $\mathcal{H}$ are defined over the same structure and differ only on
 458 their labeling. As explained in Appendix B.2, this can be obtained by taking the product of $\mathcal{T}$ and
 459 $\mathcal{H}$. Let $\mathcal{T} = \langle 2^P, \{0, 1, \dots, k\}, Q, q_0, \delta, \tau_{\mathcal{T}} \rangle$ and $\mathcal{H} = \langle 2^P, \{0, 1, \dots, m\}, Q, q_0, \delta, \tau_{\mathcal{H}} \rangle$.

460 A set $S \subseteq Q$ is in $\text{legit}(\mathcal{T}, \mathcal{H})$ if $S \in \text{legit}(\mathcal{T})$ and $S \notin \text{legit}(\mathcal{H})$. That is, there exists $i \in \{1, \dots, k\}$
 461 such that $\tau_{\mathcal{T}}(S) \subseteq \{0, i\}$, and there exists $1 \leq j < j' \leq m$ such that $\{j, j'\} \subseteq \tau_{\mathcal{H}}(S)$.

462 The problem of classification with privacy can be solved by solving the safety game $\mathcal{G}_{\mathcal{T}, \mathcal{H}}$ defined
 463 similarly to $\mathcal{G}_{\mathcal{T}}$ in Section 3, except that now V_C is the set $\text{legit}(\mathcal{T}, \mathcal{H})$ rather than $\text{legit}(\mathcal{T})$. Since
 464 $\mathcal{G}_{\mathcal{T}, \mathcal{H}}$ has $2^{O(|Q|)}$ vertices, and solving safety games can be done in linear time, the upper bound
 465 follows. $\blacktriangleleft$

466 **► Remark 11. [Richer secrets]** The results in this section can easily be extended to richer notions
 467 of privacy. Essentially, any definition that can be captured by an appropriate definition of legitimate
 468 subsets can be handled. First, *more secrets* may be added to the product, adding to the requirement
 469 of legitimate sets not to be able to classify words also according to the new secrets. Second, in our
 470 current definition, a classifier cannot classify an observed word, yet it may learn which classifications
 471 are impossible for it. We can change the definition of legitimate sets ensuring the classifier learns
 472 nothing about the secret: for every word w such that $\mathcal{H}(w) \neq 0$, there exist words $w_1, w_2, \dots, w_m$
 473 such that for every $i \in \{1, \dots, m\}$, we have that $w \sim_{\mathcal{U}} w_i$ and $\mathcal{H}(w_i) = i$. Finally, we can allow
 474 *conditional secrets*: rather than hiding the classification to all words that the secret cares about, we
 475 may require the hiding of all words that satisfy some regular property, in particular these that the
 476 secret classify to some value. $\blacktriangleleft$

477 Also in the case of privacy, the game induces a bound on the size of a correct classifier, which
 478 we show to be tight.

479 **► Theorem 12.** *For every specification $\mathcal{T}$ and secret $\mathcal{H}$, if there is a classifier that satisfies $\mathcal{T}$ and
 480 hides $\mathcal{H}$, then there is also one of size exponential in $\mathcal{T}$ and polynomial in $\mathcal{H}$. This bound is tight:
 481 For every $n \geq 1$ there is a set of signals P_n of size $O(n)$, a $(2^{P_n} / \{0, 1, 2\})$ -transducer $\mathcal{T}_n$ with $O(n)$
 482 states, and a secret $\mathcal{H}_n$ with 3 states, such that every classifier that satisfies $\mathcal{T}_n$ and hides $\mathcal{H}_n$ has
 483 at least 2^n states.*

484 **Proof.** The upper bound follows from the size of the safety game $\mathcal{G}_{\mathcal{T}, \mathcal{H}}$. For the lower bound,
 485 consider the family of specifications $\mathcal{T}_n$ defined in Theorem 8. Recall that $\mathcal{T}_n$ is over the set of
 486 signals $P_n = \{s_0, \dots, s_{n-1}, d_1, \dots, d_{n-1}\}$. By Theorem 8, every classifier that satisfies $\mathcal{T}_n$ and
 487 respects the sensing constraint $\mathcal{U}_n$ that disables the sensing of all d -signals at odd positions and
 488 allows the sensing of all other signals at all positions, has at least 2^n states.

489 Consider the secret $\mathcal{H}_n$ that induces the languages

- 490 ■ $H_1^n = (\text{True} \cdot \text{True})^* \cdot (d_1 \vee d_2 \vee \dots \vee d_{n-1})$, and
- 491 ■ $H_2^n = (\text{True} \cdot \text{True})^* \cdot (\bar{d}_1 \wedge \bar{d}_2 \wedge \dots \wedge \bar{d}_{n-1})$.



It is not hard to see that $\mathcal{H}_n$ can be defined by a transducer with three states, and that a classifier hides $\mathcal{H}_n$ iff it disables sensing all d -signals at odd positions, and so it respects $\mathcal{U}_n$. Thus, by Theorem 8, every classifier that satisfies $\mathcal{T}_n$ and hides $\mathcal{H}_n$ has at least 2^n states. ◀

► **Theorem 13.** *The problem of classification with privacy is PSPACE-hard.*

Proof. We reduce the problem of NFA universality to classification with privacy. NFA universality is PSPACE-hard already for an NFA $\mathcal{N}$ over an alphabet of size 2, with a single accepting initial state, nondeterminism of branching degree 2, and in which every non-empty word has a rejecting run. Let $\mathcal{N} = \langle \{p, \bar{p}\}, Q, q_0, \delta, \alpha \rangle$ be such an NFA. As δ has branching degree 2, we refer to it as a function $\delta : Q \times \{p, \bar{p}\} \times \{b, \bar{b}\} \rightarrow Q$, where b is a signal encoding the nondeterministic choices of $\mathcal{N}$. That is, for every $q \in Q$ and $\sigma \in \{p, \bar{p}\}$, we have that $\delta(q, \sigma) = \{\delta(q, \sigma, b), \delta(q, \sigma, \bar{b})\}$.

We construct a specification $\mathcal{T}$ and a secret $\mathcal{H}$ such that there is a classifier for $\mathcal{T}$ that does not reveal $\mathcal{H}$ iff $L(\mathcal{N}) = \{p, \bar{p}\}^*$. Essentially, $\mathcal{T}$ is over the set of signals $P = \{p, b\}$. Thus, a word $w \in (2^P)^*$ combines a word $word(w) \in \{p, \bar{p}\}^*$ along with information on how to resolve nondeterminism in a run of $\mathcal{N}$ on $word(w)$. Let $run(w)$ be the run of $\mathcal{N}$ on $word(w)$, namely the run obtained when nondeterminism is resolved by the assignments to the signal b . The specification $\mathcal{T}$ has width 2, and it induces the languages $L_1 = \text{True}^* \cdot p$ and $L_2 = \text{True}^* \cdot \bar{p}$. The secret $\mathcal{H}$ has width 2 too, and it induces the languages H_1 of words w such that $run(w)$ is accepting, and H_2 of words w such that $run(w)$ is rejecting.

In order to successfully classify words according to $\mathcal{T}$, a classifier has to constantly sense p . Recall that $\mathcal{N}$ has a rejecting run on every word in $\{p, \bar{p}\}^*$. It also has an accepting run on every word in $\{p, \bar{p}\}^*$ iff $\mathcal{N}$ is universal. Thus, a classifier that only senses p can satisfy $\mathcal{T}$ without revealing $\mathcal{H}$ iff $\mathcal{N}$ is universal. Indeed, $\mathcal{N}$ is universal iff every word $w \in (2^P)^*$ has a word $w' \in (2^P)^*$ such that $w \sim_{\{p\}} w'$ and $word(w)$ is accepting iff $word(w')$ is rejecting.

In Appendix C.4 we describe $\mathcal{T}$ and $\mathcal{H}$ in detail and prove the reduction. ◀

6 Classification with Minimal Sensing

In this section we study the problem of generating correct classifiers with minimal sensing cost. Note that two transducers may define the same mapping and have different sensing costs. For a given regular mapping $\tau : (2^P)^* \rightarrow \{1, \dots, k\}$, the sensing cost of τ , denoted $sense(\tau)$ is the minimal sensing cost required for a transducer that defines τ by a transducer. Thus, $sense(\tau) = \inf\{sense(\mathcal{C}) : \mathcal{C} \text{ defines } \tau\}$. As proven in [1], $sense(\tau)$ is attained in the minimal (size-wise) transducer that defines τ . Thus, for a given mapping τ , there is no trade-off between minimizing size and minimizing sensing, and a minimally-sensing transducer that defines τ can be found in time polynomial in some transducer that defines τ . In the classification problem, however, the mapping τ is not given, as we have freedom with respect to words that the specification $\mathcal{T}$ maps to 0. In the case of size minimization, going over all correct mappings leads to an algorithm in NP, and the problem is NP-complete. As we show, the case of sensing minimization is more involved.

6.1 Trade-off Size and Sensing in Separation

As demonstrated in Example 1, minimal sensing may be attained by a classifier that is not of minimal size. Here, we strengthen the size-sensing trade-off in two ways. We show that it is unbounded and that a minimally-sensing classifier may be exponentially bigger than its specification.

► **Theorem 14.** *For every $n \geq 2$, there is a $(2^{\{p\}}/\{0, 1, 2\})$ -transducer $\mathcal{T}_n$ with $O(n)$ states such that there is a two-state classifier for $\mathcal{T}_n$, every two-state classifier for $\mathcal{T}_n$ has sensing cost 1, and there is a $2n$ -state classifier for $\mathcal{T}_n$ with sensing cost $\frac{1}{n}$.*

Proof. We generalize the transducer from Example 1 so that $\mathcal{T}_n$ defines the languages $L_1^n = (\text{True}^n)^* \cdot p^n$ and $L_2^n = (\text{True}^n)^* \cdot \bar{p}^n$. (Note that Example 1 corresponds to the case $n = 2$).



In Appendix C.5, we describe $\mathcal{T}_n$ in detail. It is not hard to see that the two-state classifier $\mathcal{C}_1$ from Example 1 can serve as a classifier for $\mathcal{T}_n$ for all $n \geq 2$. Indeed $\mathcal{C}_1$ maps to 1 exactly all words ending in p , and so it is still the case that all words in L_1^n are mapped to 1 and all words in L_2^n are mapped to 2. In the appendix, we prove that every two-state classifier with $\text{sense}(\mathcal{C}) < 1$, namely a classifier that does not sense p in at least one of its states, cannot satisfy $\mathcal{T}_n$. Essentially, such a classifier must contain a problematic cycle.

Finally, a classifier with $2n$ -states can map to 1 exactly all words in $(\text{True}^n)^*.p.\text{True}^{n-1}$, sensing p only once every n transitions, thus having sensing cost $\frac{1}{n}$. In Appendix C.5, we describe such a classifier in detail. ◀

We continue and prove that a minimally-sensing classifier for a given language classification $\mathcal{T}$ may be exponentially bigger than $\mathcal{T}$. Specifically, we prove the following theorem.

► **Theorem 15.** *For every $n \geq 1$, there is a specification $\mathcal{T}_n$ with $O(n)$ states such that $\mathcal{T}_n$ can be satisfied by a classifier with sensing cost $\frac{3n-1}{2}$, yet every classifier $\mathcal{C}_n$ that satisfies $\mathcal{T}_n$ and has $\text{sense}(\mathcal{C}_n) \leq \frac{3n-1}{2}$ needs at least 2^n states.*

Proof. Recall the family of transducers $\mathcal{T}_n$ defined in the proof of Theorem 8. In Appendix C.6, we prove (Theorem 20), that the classifier $\mathcal{C}_n^*$ described there has sensing cost $\frac{3n-1}{2}$. Then, in Theorem 21, we prove that every classifier $\mathcal{C}_n$ for $\mathcal{T}_n$ with sensing cost $\frac{3n-1}{2}$ has at least 2^n states. ◀

6.2 Finding A Classifier with Minimal Expected Sensing

In this section we study the problem of generating a classifier with minimal sensing cost. Recall the 2-player game $\mathcal{G}_{\mathcal{T}}$ that embodies possible classifiers for a given specification $\mathcal{T}$. A 1.5-player game is similar to a 2-player game, except that now Environment is probabilistic, and we refer to it as *Nature*. In addition, in order to reason about the expected sensing, we augment the states of the game by a cost function, describing the number of signals sensed in each chose of Classifier, and study the *mean-payoff* of plays in the game.

Formally, given $\mathcal{T} = \langle 2^P, \{0, \dots, k\}, Q, q_0, \delta, \tau \rangle$, we define the 1.5-player safety mean-payoff game $\mathcal{M}_{\mathcal{T}} = \langle V_C, V_E, v_0, v_{\perp}, E, P, \text{cost} \rangle$ as follows. As in $\mathcal{G}_{\mathcal{T}}$, we have that $V_C = \text{legit}(\mathcal{T})$, $V_E = \text{legit}(\mathcal{T}) \times 2^P$, $v_0 = \{q_0\}$, and for each $S \in \text{legit}(\mathcal{T})$ and $P' \subseteq P$, we have that $\langle S, \langle S, P' \rangle \rangle \in E$. In addition $P : V_E \times (V_C \cup \{v_{\perp}\}) \rightarrow [0, 1]$ is a probabilistic transition function, defining for every two states $s \in V_E$ and $s' \in V_C \cup \{v_{\perp}\}$, the probability of Nature moving the token from s to s' . Since we assume a uniform distribution on valuations to the input signals, the function P is defined as follows. Consider a state $\langle S, P' \rangle \in V_E$.

■ For every set $S' \in \text{legit}(\mathcal{T})$, we have that $P(\langle S, P' \rangle, S') = \frac{|\{\sigma \in 2^P : \text{succ}(S, P', \sigma) = S'\}|}{2^{|P|}}$. That is, the probability of transitioning from $\langle S, P' \rangle$ to S' is the fraction of assignments $\sigma \in 2^P$ such that $\text{succ}(S, P', \sigma) = S'$.

■ In addition, $P(\langle S, P' \rangle, v_{\perp}) = \frac{|\{\sigma \in 2^P : \text{succ}(S, P', \sigma) \notin \text{legit}(\mathcal{T})\}|}{2^{|P|}}$. That is, the probability of transitioning from $\langle S, P' \rangle$ to $v_{\perp}$ is the fraction of assignments $\sigma \in 2^P$ such that $\text{succ}(S, P', \sigma)$ is not in $\text{legit}(\mathcal{T})$.

Note that in both cases, the probability may be 0. Finally, the cost function $\text{cost} : V_E \rightarrow \{0, \dots, |P|\}$ is defined for every state in V_E by $\text{cost}(\langle S, P' \rangle) = |P'|$.

A positional strategy $f : V_C \rightarrow V_E$ for Classifier induces a Markov Chain $\mathcal{M}_{\mathcal{T}}^f = \langle V_C \cup \{v_{\perp}\}, \{q_0\}, P_f \rangle$, where for every two states $S, S' \in V_C$, we have that $P_f(S, S') = P(\langle S, f(S) \rangle, S')$. In addition, $P_f(S, v_{\perp}) = P(\langle S, f(S) \rangle, v_{\perp})$. That is, $\mathcal{M}_{\mathcal{T}}^f$ describes the expected behavior of $\mathcal{M}_{\mathcal{T}}$ when Classifier follows f . The *cost* of f is then the expected average cost of a random walk in $\mathcal{M}_{\mathcal{T}}^f$. Again, the cost of f can be calculated from the limiting distribution of $\mathcal{M}_{\mathcal{T}}^f$ (see e.g., [25]). Let $\text{cost}(\mathcal{M}_{\mathcal{T}}^f) = \inf\{\text{cost}(f) : f \text{ is a positional strategy for } \mathcal{M}_{\mathcal{T}} \text{ that avoids } v_{\perp}\}$. That is, $\text{cost}(\mathcal{M}_{\mathcal{T}}^f)$ is the expected cost of a game played on $\mathcal{M}_{\mathcal{T}}^f$ in which Classifier uses an optimal policy that avoids $v_{\perp}$. By [7], $\text{cost}(\mathcal{M})$ the restriction to positional strategies does not increase the cost, and $\text{cost}(\mathcal{M}_{\mathcal{T}}^f)$ can be computed in polynomial time.

584 Since $\text{legit}(\mathcal{T})$ and hence $\mathcal{M}_{\mathcal{T}}$ is of size exponential in $\mathcal{T}$, upper bounds to the problem and to
 585 the size of the classifier follow:

586 ► **Theorem 16.** *The problem of finding a classifier with minimal sensing can be solved in EXPTIME.*
 587 *Moreover, for a specification with n states, such a classifier needs at most 2^n states.*

588 Theorem 15 provides a matching lower bound for the size of the classifier. For the decision
 589 problem, we are able to show a PSPACE lower bound.

590 ► **Theorem 17.** *Given a specification $\mathcal{T}$ and a number $\gamma > 0$, it is PSPACE-hard to decide whether*
 591 *there is a classifier for $\mathcal{T}$ with sensing cost strictly smaller than γ .*

592 **Proof.** We describe a reduction from the universality problem for NFAs. That is, given an NFA
 593 $\mathcal{N}$ over an alphabet Σ , we describe $\gamma \geq 1$ and a specification $\mathcal{T}$ of two languages L_1 and L_2 , such
 594 that $\mathcal{T}$ can be constructed from $\mathcal{N}$ in polynomial time, and $L(\mathcal{N}) \neq \Sigma^*$ iff there is a classifier for $\mathcal{T}$
 595 with sensing strictly smaller than γ .

596 Since the universality problem is PSPACE-hard already for NFAs with a single initial state,
 597 no empty states, and a nondeterminism branching degree exactly 2 [22], we assume that $\mathcal{N} =$
 598 $\langle Q, \Sigma, \delta, q_0, \alpha \rangle$ is such that no state in $\mathcal{N}$ is empty and the transition function δ gets as a parameter
 599 also an index that resolves the nondeterminism. Thus, as in the proof of Theorem 13, we assume
 600 that $\delta : Q \times \Sigma \times \{b, \bar{b}\} \rightarrow Q$, where b is a signal encoding the nondeterministic choices of $\mathcal{N}$.

601 Before describing the languages L_1 and L_2 , let us describe the set P of signals that defines
 602 their alphabet. Let P_{Σ} be such that $\Sigma = 2^{P_{\Sigma}}$. We define $P = P_{\Sigma} \cup \{b, r_1, r_2, s\}$, with the following
 603 intuition. Words in $(2^P)^*$ correspond to words over Σ , extended with information about resolving
 604 of nondeterminism (the signal b), resetting of the run (the signals r_1 and r_2), and choosing between
 605 L_1 and L_2 (the signal s).

606 Let $P_{base} = P_{\Sigma} \cup \{r_1, r_2\}$. Also, we say that an assignment a is a *reset assignment* if at least
 607 one of the reset signals (r_1 or r_2) are assigned **True**. Otherwise, a is a *simulation assignment*.

608 For a word $w \in (2^P)^*$, let $v = a_1 \dots a_k$ be the maximal suffix of w consisting of simulation
 609 assignments only. For $1 \leq j \leq k$, let $(\sigma^j, b^j, r_1^j, r_2^j, s^j)$ be the projection of a_j on P_{Σ} , b , r_1 , r_2 , and
 610 s , respectively. Let $\text{word}(w) = \sigma^1 \dots \sigma^k \in \Sigma^*$, and let $\text{run}(w) \in Q^*$ be the run of $\mathcal{N}$ on $\text{word}(w)$
 611 when it resolves its nondeterministic choices according to the assignments to the signal b . Formally,
 612 $\text{run}(w) = q_0, q_1, \dots, q_k$, where for all $1 \leq j \leq k$, we have that $q_j = \delta(q_{j-1}, \sigma^j, b^j)$. Finally, let
 613 $\text{last}(w) = q_k$ be the last state in $\text{run}(w)$, namely the state that $\mathcal{N}$ reaches on its run on $\text{word}(w)$
 614 when it resolves its nondeterministic choices according to the assignments to the signal b .

615 The languages $L_1, L_2 \subseteq (2^P)^+$ are languages of non-empty words defined as follows. Consider a
 616 word of the form $w \cdot a$, for $w \in (2^P)^*$ and $a \in 2^P$.

617 ■ If a is a reset assignment, then $w \cdot a \in L_1 \cup L_2$, and classification depends on the parity of the
 618 signals in P_{base} to which a assigns **True**: If $|a \cap P_{base}|$ is odd, then $w \cdot a \in L_1$, and if $|a \cap P_{base}|$
 619 is even, then $w \cdot a \in L_2$.

620 ■ If a is a simulation assignment, then $w \cdot a \in L_1 \cup L_2$ iff $\text{last}(w) \in \alpha$, and classification depends
 621 on the assignment to b in a : If $b \in a$, then $w \cdot a \in L_1$, and if $b \notin a$, then $w \cdot a \in L_2$.

622 Note that in the first case, classification is independent of w , whereas in the second case, the
 623 condition about membership in α is independent of a .

624 We first describe a $(2^P / \{0, 1, 2\})$ -transducer $\mathcal{T}$ that defines the languages L_1 and L_2 and is
 625 polynomial in $\mathcal{N}$. We define $\mathcal{T} = \langle 2^P, \{0, 1, 2\}, Q', q'_0, \delta', \tau \rangle$ as follows. The state space $Q' =$
 626 $Q \times \{0, 1, 2\}$ consists of three copies of Q , and $\mathcal{T}$ starts in the state $q'_0 = \langle q_0, 0 \rangle$. Intuitively, $\mathcal{T}$
 627 simulates runs of $\mathcal{N}$ on words over $2^{P_{\Sigma}}$, using the information from the other signals in order to
 628 reset the runs, resolve nondeterminism, and choose among Copies 1 and 2, which is needed whenever
 629 the next assignment is a reset assignment, or when the next assignment is a simulation assignment

630 and the current state is accepting. Formally, for a state $\langle q, c \rangle \in Q'$ and letter $a \in 2^P$, we define

$$631 \quad \delta'(\langle q, c \rangle, a) = \begin{cases} \langle q_0, 1 \rangle & \text{if } a \text{ is a reset assignment and } |a \cap P_{base}| \text{ is odd} \\ \langle q_0, 2 \rangle & \text{if } a \text{ is a reset assignment and } |a \cap P_{base}| \text{ is even} \\ \langle \delta(q, \sigma, i), 0 \rangle & \text{if } a \text{ is a simulation assignment and } q \notin \alpha \\ \langle \delta(q, \sigma, i), 1 \rangle & \text{if } a \text{ is a simulation assignment } q \in \alpha, \text{ and } b \in a \\ 632 \quad \langle \delta(q, \sigma, i), 2 \rangle & \text{if } a \text{ is a simulation assignment } q \in \alpha, \text{ and } b \notin a \end{cases}$$

633 Finally, $\tau(\langle q, c \rangle) = c$.

634 It is not hard to see that $\mathcal{T}$ indeed defines L_1 and L_2 . Let $\mathcal{M}_{\mathcal{T}}$ be the 1.5-player game constructed
635 from $\mathcal{T}$. In Appendix C.7, we prove that $\mathcal{N}$ is universal iff $\text{sense}(\mathcal{M}_{\mathcal{T}}) \geq |P_{base}| + 1$. The idea of the
636 proof is as follows. First, we consider a classifier $\mathcal{C}$ that senses all signals in P_{base} at all times, never
637 senses b , and senses s only when $\text{word}(w)$ is in $L(\mathcal{N})$. The classifier $\mathcal{C}$ is correct, as if the value of s
638 is relevant after reading a word w , then $\text{word}(w) \in L(\mathcal{N})$. Note that $\mathcal{C}$ has sensing cost of exactly
639 $|P_{base}| + 1$ when $\mathcal{N}$ is universal, and strictly smaller than $|P_{base}| + 1$ when $\mathcal{N}$ is not universal. We
640 then prove that $\mathcal{C}$ is optimal. This part involves a careful analysis of $\text{sense}(\mathcal{M}_{\mathcal{T}})$, and essentially
641 follows from the fact that sensing b (aiming to reduce the uncertainty due to nondeterminism) is
642 never beneficial: the savings from knowing which nondeterministic choice is made are outweighed
643 by the cost of sensing b itself. Thus, $\mathcal{N}$ is non-universal iff a classifier can achieve sensing cost
644 strictly below $|P_{base}| + 1$. ◀

645 7 Discussion

646 We introduced and studied *classification under uncertainty*, where we seek classifiers that satisfy
647 a given language specification over an alphabet 2^P despite uncertainty about the values of some
648 signals in P .

649 We studied three sources of uncertainty and showed that they all may require a classifier for
650 a specification $\mathcal{T}$ to be of size exponential in $|\mathcal{T}|$. This exponential blow-up is similar to the one
651 exhibited in other games with uncertainty [26]. In particular, in *synthesis with privacy* [18], one
652 seeks transducers that satisfy a given specification while keeping the truth value of given secrets
653 unknown, and in *synthesis with minimal sensing* [1], one seeks transducers that satisfy a given
654 specification with minimal sensing of input signals. Both problems are EXPTIME-complete already
655 for specifications given by deterministic automata. Since correct classification can be specified by a
656 deterministic automaton, the problems are strongly related, and the game $\mathcal{G}_{\mathcal{T}}$ described in Section 3
657 is indeed similar to the game-theoretic approach used in [1, 18].

658 Classification and synthesis, however, have a conceptual difference, and the tight complexity
659 of classification with privacy and with minimal sensing is unfortunately left open. Moreover, we
660 found the lower bounds in Theorems 8, 15, and 17, much more complicated than these used in
661 the synthesis case. In order to see the challenge in classification, recall (Theorem 7) that deciding
662 whether $\mathcal{T}$ can be satisfied by a classifier that respects a given sensing constraint $\mathcal{U}$ can be done
663 in NLOGSPACE. This is not the case for synthesis: deciding whether a specification given by a
664 deterministic automaton $\mathcal{A}$ can be realized by a transducer that respect a given sensing constraint
665 $\mathcal{U}$ is EXPTIME-hard [2]! The challenge in lower bounds for classification problems has to do with
666 the fact that in classification, bad strategies of Classifier in the game $\mathcal{G}_{\mathcal{T}}$ have a relatively immediate
667 effect, which can be detected by following two (rather than all) futures of the game. Thus, we still
668 have an element of alternation, but it is weaker than the one that takes us from NLOGSPACE to
669 PTIME in reachability [16]. When $\mathcal{U}$ is not given, we cannot avoid exploring all the subsets of the
670 state space of $\mathcal{T}$, hence the PSPACE lower bound, but the acrobatics required in order to generate
671 instances that generate all these subsets is much higher, and the amount of alternation that is
672 needed in for their exploration is still open, inducing the gap between PSPACE and EXPTIME.



- 1 S. Almagor, D. Kuperberg, and O. Kupferman. Sensing as a complexity measure. *Int. J. Found. Comput. Sci.*, 30(6-7):831–873, 2019.
- 2 K. Chatterjee and L. Doyen. The complexity of partial-observation parity games. In *Proc. 16th Int. Conf. on Logic for Programming Artificial Intelligence and Reasoning*, pages 1–14. Springer, 2010.
- 3 A. Cimatti, C. Tian, and S. Tonetta. Assumption-based runtime verification with partial observability and resets. In *19th International Conference on Runtime Verification*, Lecture Notes in Computer Science, pages 165–184. Springer, 2019.
- 4 G. Cugola and A. Margara. Processing flows of information: From data stream to complex event processing. *ACM Comput. Surv.*, 44(3), 2012.
- 5 W. Czerwinski, S. Lasota, R. Meyer, S. Muskalla, K.N. Kumar, and P. Saivasan. Regular separability of well-structured transition systems. In *Proc. 29th Int. Conf. on Concurrency Theory*, volume 118 of *LIPICs*, pages 35:1–35:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- 6 W. Czerwinski, W. Martens, and T. Masopust. Efficient separability of regular languages by subsequences and suffixes. In *Proc. 40th Int. Colloq. on Automata, Languages, and Programming*, volume 7966 of *Lecture Notes in Computer Science*, pages 150–161. Springer, 2013.
- 7 L. de Alfaro. *Formal Verification of Probabilistic Systems*. PhD thesis, Stanford University, 1997.
- 8 S. Dziembowski, M. Jurdzinski, and I. Walukiewicz. How much memory is needed to win infinite games. In *Proc. 12th ACM/IEEE Symp. on Logic in Computer Science*, pages 99–110, 1997.
- 9 A. Ferrando and V. Malvone. Runtime verification via rational monitor with imperfect information. *ACM Trans. on Software Engineering and Methodology*, 35(3):74:1–74:29, 2026.
- 10 B. Finkbeiner. Logics and algorithms for hyperproperties. *SIGLOG News*, 5(1):31–45, 2018.
- 11 G. Gange, P. Ganty, and P.J. Stuckey. Fixing the state budget: Approximation of regular languages with small dfas. In *15th Int. Symp. on Automated Technology for Verification and Analysis*, volume 10482 of *Lecture Notes in Computer Science*, pages 67–83. Springer, 2017.
- 12 E. M. Gold. Complexity of automaton identification from given data. *Information and Control*, 37(3):302–320, 1978.
- 13 A. E. Goodloe and K. Havelund. High-integrity runtime verification. *Computer*, 57(4):37–45, 2024.
- 14 C. Grinstead and J. Laurie Snell. 11:markov chains. In *Introduction to Probability*. American Mathematical Society, 1997.
- 15 M. Heule and S. Verwer. Exact DFA identification using SAT solvers. In *Grammatical Inference: Theoretical Results and Applications, 10th International Colloquium*, volume 6339 of *Lecture Notes in Computer Science*, pages 66–79. Springer, 2010.
- 16 N. Immerman. Number of quantifiers is better than number of tape cells. *Journal of Computer and Systems Science*, 22(3):384–406, 1981.
- 17 O. Kupferman, N. Lavee, and S. Sickert. Certifying DFA bounds for recognition and separation. *Innov. Syst. Softw. Eng.*, 18(3):405–416, 2022.
- 18 O. Kupferman and O. Leshkowitz. Synthesis of privacy-preserving systems. In *Proc. 42nd Conf. on Foundations of Software Technology and Theoretical Computer Science*, volume 250 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 42:1–42:23, 2022.
- 19 O. Kupferman, O. Leshkowitz, and N. Shamash Halevy. Synthesis with privacy against an observer. *Logical Methods in Computer Science*, 21(3), 2025.
- 20 M. Leucker, C. Sánchez, T. Scheffel, M. Schmitz, and D. Thoma. Runtime verification for timed event streams with partial information. In *19th International Conference on Runtime Verification*, Lecture Notes in Computer Science, pages 273–291. Springer, 2019.
- 21 M. Leucker and C. Schallhart. A brief account of runtime verification. *The Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- 22 A.R. Meyer and L.J. Stockmeyer. The equivalence problem for regular expressions with squaring requires exponential space. In *Proc. 13th IEEE Symp. on Switching and Automata Theory*, pages 125–129, 1972.
- 23 T. Place and M. Zeitoun. Separating regular languages with first-order logic. *Log. Methods Comput. Sci.*, 12(1), 2016.



- 725 **24** A. Pnueli. The temporal semantics of concurrent programs. *Theoretical Computer Science*, 13:45–60,
726 1981.
- 727 **25** M.L. Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley &
728 Sons, 2014.
- 729 **26** J.H. Reif. The complexity of two-player games of incomplete information. *Journal of Computer and*
730 *Systems Science*, 29:274–301, 1984.
- 731 **27** B.A. Trakhtenbrot and Y.M. Barzdin. *Finite Automata*. North Holland, 1973.
- 732 **28** M.Y. Vardi and P. Wolper. Reasoning about infinite computations. *Information and Computation*,
733 115(1):1–37, 1994.

734 **A** Calculating the Sensing Cost of a Transducer

735 Consider a $(2^P/\Sigma_O)$ -transducer $\mathcal{T} = \langle 2^P, \Sigma_O, Q, q_0, \delta, \tau \rangle$. Recall that for a finite word $w \in (2^P)^*$,
736 we have that $\text{sense}(w)$ is the sensing cost of the run of $\mathcal{T}$ on w , and $\text{sense}(\mathcal{T})$ is the expected
737 sensing cost of words of length that tends to infinity.

Thus,

$$\text{sense}(\mathcal{T}) = \lim_{m \rightarrow \infty} |\Sigma|^{-m} \sum_{w: |w|=m} \text{sense}(w).$$

738 A-priori, it is not clear that $\text{sense}(\mathcal{T})$ is well defined, i.e., that the limit above always exists. As
739 detailed below, the limit does always exist, and can be found by analyzing the limiting distribution
740 of a random walk on $\mathcal{T}$.

741 Consider a directed graph $G = \langle V, E \rangle$. A *strongly connected component* (SCC) of G is a maximal
742 (with respect to containment) set $C \subseteq V$ such that for all $x, y \in C$, there is a path from x to y . An
743 SCC (or state) is *ergodic* if no other SCC is reachable from it, and is *transient* otherwise.

744 A transducer $\mathcal{T} = \langle 2^P, \Sigma_O, Q, q_0, \delta, \tau \rangle$ induces a directed graph $\mathcal{G}_{\mathcal{T}} = \langle Q, E \rangle$ in which $\langle q, q' \rangle \in E$
745 iff there is a letter $\sigma \in 2^P$ such that $q' = \delta(q, \sigma)$. When we talk about the SCCs of $\mathcal{T}$, we refer to
746 those of $\mathcal{G}_{\mathcal{T}}$.

747 Let $\mathcal{E}$ be the set of ergodic SCCs of $\mathcal{T}$. Consider an ergodic SCC $C \in \mathcal{E}$. Let P_C be the matrix
748 describing the probability of transitions in C . Thus, the rows and columns of P_C are associated
749 with states in C , and the value in coordinate q, q' is $P_{q, q'}$. By [14], there is a unique probability
750 vector $\pi_C \in [0, 1]^C$ such that $\pi_C P_C = \pi_C$. This vector describes the *stationary distribution* of C :
751 for all $q \in C$ it holds that $\pi_C(q) = \lim_{m \rightarrow \infty} \frac{O_m^C(q)}{m}$ almost surely, where $O_m^C(q)$ is the number of
752 occurrences of q in a run of $M_{\mathcal{T}}$ of length m that starts from some arbitrary state in C [14]. Thus,
753 intuitively, $\pi_C(q)$ is the proportion of time that a long run that begins in C spends in q . Note that
754 π_C is the unique stationary distribution of C . In order to extend the distribution to the entire
755 Markov chain of $\mathcal{T}$, we have to take into account the probability of reaching each of the ergodic
756 components. The *SCC-reachability distribution* of $\mathcal{T}$ is the function $\rho: \mathcal{E} \rightarrow [0, 1]$ that maps each
757 ergodic SCC C of $\mathcal{T}$ to the probability that $M_{\mathcal{T}}$ eventually reaches C , starting from the initial
758 state.

759 The *limiting distribution* $\pi: Q \rightarrow [0, 1]$ is now defined by $\pi(s) = 0$, if s is transient, and
760 $\pi(s) = \pi_C(s) \cdot \rho(C)$, if s is in some $C \in \mathcal{E}$. By [14], the limiting distribution can be computed
761 in polynomial time by solving a system of linear equations. Intuitively, the limiting distribution
762 at state q describes the expected proportion of time that a long run of $M_{\mathcal{T}}$ that starts in q_0
763 spends in q . Formally, by the Strong Law of Large Numbers for Markov chains, we have that
764 $\pi(s) = \lim_{m \rightarrow \infty} \frac{\mathbb{E}[O_m(s)]}{m}$, where $O_m(s)$ be the number of occurrences of s in a run of length m of
765 $M_{\mathcal{T}}$ that starts in s_0 . Note that when the Markov chain has a single ergodic component, then the
766 limiting distribution coincides with the stationary distribution of that component. That is, for all
767 $q \in Q$, we have $\pi(q) = \pi_C(q)$ where C is the unique ergodic component of $M_{\mathcal{T}}$. Moreover, in that
768 case, by the Ergodic Theorem for Markov chains it holds that $\pi(q) = \lim_{m \rightarrow \infty} \frac{O_m(q)}{m}$ almost surely,
769 and not only $\pi(q) = \mathbb{E}[\lim_{m \rightarrow \infty} \frac{O_m(q)}{m}] = \lim_{m \rightarrow \infty} \frac{\mathbb{E}[O_m(q)]}{m}$ in expectation as in the general case.



As proven in [1], the sensing cost of a transducer $\mathcal{T}$ can be expressed also with respect to its limiting distribution, which enables a calculation of $\text{sense}(\mathcal{T})$ in polynomial time. Formally, if the limiting distribution of $M_{\mathcal{T}}$ is $\pi : Q \rightarrow [0, 1]$, then, $\text{sense}(\mathcal{T}) = \sum_{q \in Q} \pi(q) \cdot \text{sense}(q)$.

B Remarks

B.1 Describing several languages by a single transducer

Describing k disjoint languages by a $(\Sigma/\{0, 1, \dots, k\})$ -transducer rather than by k DFWs leads to a tighter complexity analysis, in the following sense. Given DFWs $\mathcal{A}_i = \langle \Sigma, Q_i, q_0^i, \delta_i, \alpha_i \rangle$ for the underlying languages L_i , a k -language specification $\mathcal{T}$ that induces $L_1, \dots, L_k$ can be defined on top of the product, thus $\mathcal{T} = \langle \Sigma, Q_1 \times \dots \times Q_k, \langle q_0^1, \dots, q_0^k \rangle, \delta, \tau \rangle$, where for all $\langle q_1, \dots, q_k \rangle \in Q_1 \times \dots \times Q_k$ and $\sigma \in \Sigma$, we have that $\delta(\langle q_1, \dots, q_k \rangle, \sigma) = \langle \delta_1(q_1, \sigma), \dots, \delta_k(q_k, \sigma) \rangle$. The fact the languages are disjoint guarantees that for every state $\langle q_1, \dots, q_k \rangle \in Q_1 \times \dots \times Q_k$, there is at most one $i \in \{1, \dots, k\}$ such that $q_i \in \alpha_i$. If such $i \in \{1, \dots, k\}$ exists, then $\tau(\langle q_1, \dots, q_k \rangle) = i$, and otherwise $\tau(\langle q_1, \dots, q_k \rangle) = 0$. Since a classifier with full visibility can be defined on top of $\mathcal{T}$, we find an analysis that refers to $\mathcal{T}$ tighter in the study of uncertainty. In particular, when k is a constant, $\mathcal{T}$ is polynomial in the underlying $\mathcal{A}_i$'s.

B.2 Composing the specification and the secret

The assumption that $\mathcal{T}$ and $\mathcal{H}$ are defined over the same structure can be obtained by taking the product of $\mathcal{T}$ and $\mathcal{H}$. Formally, given $\mathcal{T} = \langle 2^P, \{0, 1, \dots, k\}, Q, q_0, \delta, \tau_{\mathcal{T}} \rangle$ and $\mathcal{H} = \langle 2^P, \{0, 1, \dots, m\}, Q', q'_0, \delta', \tau_{\mathcal{H}} \rangle$, we define the product $\mathcal{T} \times \mathcal{H} = \langle 2^P, \{0, \dots, k\} \times \{0, \dots, m\}, Q \times Q', \langle q_0, q'_0 \rangle, \delta'', \tau \rangle$, which is a $(2^P/\{0, \dots, k\} \times \{0, \dots, m\})$ -transducer defined as follows. The transition function δ'' is the product of δ and δ' , i.e., for every $\langle q, q' \rangle \in Q \times Q'$ and $\sigma \in 2^P$, we have that $\delta''(\langle q, q' \rangle, \sigma) = \langle \delta(q, \sigma), \delta'(q', \sigma) \rangle$. The labeling function τ is defined by $\tau(\langle q, q' \rangle) = \langle \tau_{\mathcal{T}}(q), \tau_{\mathcal{H}}(q') \rangle$ for every $\langle q, q' \rangle \in Q \times Q'$. It is not hard to see that by projecting the labels of $\mathcal{T} \times \mathcal{H}$ to the first and second components, we can recover $\mathcal{T}$ and $\mathcal{H}$, respectively, yet with a shared structure.

C Missing Proofs

C.1 Missing details in the proof of Lemma 2

Let $\mathcal{T} = \langle 2^P, \{0, \dots, k\}, Q, q_0, \delta, \tau \rangle$ and $\mathcal{C} = \langle 2^P, \{1, \dots, k\}, C, c_0, \eta, \kappa \rangle$. We need to verify that for every word $w \in (2^P)^*$, if $\mathcal{T}(w) \in \{1, \dots, k\}$, then $\mathcal{C}(w) = \mathcal{T}(w)$.

We claim that if there is a word $w \in (2^P)^*$ such that $\mathcal{T}(w) \in \{1, \dots, k\}$ and $\mathcal{C}(w) \neq \mathcal{T}(w)$, then there is also such a word of length at most $|Q| \cdot |C|$. Indeed, a word $w = \sigma_1 \dots \sigma_n$, with $n > |Q| \cdot |C|$, must include two positions $1 \leq i_1 < i_2 \leq |Q| \cdot |C| + 1$ such that $\delta(q_0, \sigma_1 \dots \sigma_{i_1}) = \delta(q_0, \sigma_1 \dots \sigma_{i_2})$ and $\eta(q_0, \sigma_1 \dots \sigma_{i_1}) = \eta(q_0, \sigma_1 \dots \sigma_{i_2})$, and can thus be shortened (by removing the $\sigma_{i_1+1} \dots \sigma_{i_2}$ infix) to a shorter word with the same mappings by $\mathcal{T}$ and $\mathcal{C}$. Repeating this results in a word of length at most $|Q| \cdot |C|$.

Accordingly, an NLOGSPACE Turing machine can guess a word over $(2^P)^*$, maintaining in each iteration the two states of the runs on $\mathcal{T}$ and $\mathcal{C}$ on it, and a counter. The machine accepts if the states do not agree on a label in $\{1, \dots, k\}$, and rejects when the counter exceeds $|Q| \cdot |C|$.

Hardness in NLOGSPACE is by an easy reduction from non-reachability in a directed graph: Given a directed graph $G = \langle V, E \rangle$ and two vertices $s, t \in V$, let $\Sigma = V$, and let $\mathcal{T}$ map to 1 all words in $s \cdot V^* \cdot t$ that correspond to paths in G and map to 2 all words in $s \cdot (V \setminus \{t\})^*$. Then, let $\mathcal{C}$ map to 2 all words in $s \cdot V^*$. Clearly, $\mathcal{C}$ satisfies $\mathcal{T}$ iff there is no path from s to t in G . Also, both $\mathcal{T}$ and $\mathcal{C}$ can be generated in logarithmic space from G .



C.2 Missing details in the proof of Lemma 6

The implication (3) $\rightarrow$ (2) is immediate, and (2) $\rightarrow$ (3) is proved in the body of the proof. It remains to show that (1) $\leftrightarrow$ (2). By Theorem 5, there is no classifier for $\mathcal{T}$ that respects $\mathcal{U}$ iff Classifier loses the game $\mathcal{G}_{\mathcal{T}\mathcal{U}}$. Since Classifier is forced to follow $\mathcal{U}$, this game is equivalent to a reachability game for Environment: Classifier loses iff $v_\perp$ is reachable from the initial position.

Moreover, as argued in Section 4, every position of Classifier in $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ is of the form $\langle S, r \rangle$ for some $S \in \text{legit}(\mathcal{T})$ and $r \in R$. From such a position, Classifier can only follow $\mathcal{U}$ and thus sense the set $\mu(r)$. Then, for every letter $\sigma \in 2^P$ chosen by the Environment, the successor position is $\langle S', r' \rangle$ where $S' = \text{succ}(S, \mu(r), \sigma)$ and $r' = \rho(r, \sigma)$, unless $S' \notin \text{legit}(\mathcal{T})$, in which case the game moves to $v_\perp$. In addition, since moves of the Environment depend only on the $\sim_{\mu(r)}$ -equivalence class of the chosen letter σ , we have that for every $\sigma, \sigma' \in 2^P$ such that $\sigma \sim_{\mu(r)} \sigma'$, the successor position is the same. Hence, words that are $\sim_{\mathcal{U}}$ -equivalent induce the same path in $\mathcal{G}_{\mathcal{T}\mathcal{U}}$.

Consider a word $w = \sigma_1 \cdots \sigma_m$ and the corresponding sequence $\langle S_0, r_0 \rangle, \dots, \langle S_m, r_m \rangle$ of pairs in $2^Q \times R$ defined inductively as follows. Let $S_0 = \{q_0\}$, and for every $1 \leq i \leq m$, let $S_i = \text{succ}(S_{i-1}, \mu(r_{i-1}), \sigma_i)$ and $r_i = \rho(r_{i-1}, \sigma_i)$. We prove that for every $0 \leq i \leq m$, we have that $S_i = \delta(q_0, [w]_{\mathcal{U}})$, where $[w]_{\mathcal{U}}$ denotes the $\sim_{\mathcal{U}}$ -equivalence class of w and $\delta(q_0, [w]_{\mathcal{U}})$ denotes the set of states reached from q_0 by words that are $\sim_{\mathcal{U}}$ -equivalent to w . The proof is by induction on i . For $i = 0$, we have that $S_0 = \{q_0\}$, and the claim holds as ε is the only word of length 0, so $[\varepsilon]_{\mathcal{U}} = \{\varepsilon\}$ and the claim holds trivially. Assume that the claim holds for $i - 1 \geq 0$. We prove that it holds for i . By definition $S_i = \text{succ}(S_{i-1}, \mu(r_{i-1}), \sigma_i)$, and by the induction hypothesis, we have that $S_{i-1} = \delta(q_0, [w]_{\mathcal{U}})$. Hence,

$$\begin{aligned} S_i &= \text{succ}(S_{i-1}, \mu(r_{i-1}), \sigma_i) \\ &= \{ \delta(s, \sigma') : s \in S_{i-1}, \text{ and } \sigma' \sim_{\mu(r_{i-1})} \sigma_i \} \\ &= \{ \delta(q_0, w' \cdot \sigma') : w' \sim_{\mathcal{U}} \sigma_1 \cdots \sigma_{i-1}, \text{ and } \sigma' \sim_{\mu(r_{i-1})} \sigma_i \} \\ &= \{ \delta(q_0, w') : w' \sim_{\mathcal{U}} \sigma_1 \cdots \sigma_i \} \\ &= \delta(q_0, [w]_{\mathcal{U}}) \end{aligned}$$

Clearly, w witnesses a path in $\mathcal{G}_{\mathcal{T}\mathcal{U}}$ that ends in $v_\perp$ iff the last position $\langle S_m, r_m \rangle$ is such that $S_m \notin \text{legit}(\mathcal{T})$. Hence, $v_\perp$ is reachable from the initial position iff there is a word $w \in (2^P)^*$ such that $\delta(q_0, [w]_{\mathcal{U}}) \notin \text{legit}(\mathcal{T})$. The latter holds iff there are words $w_1, w_2 \in [w]_{\mathcal{U}}$ such that $\mathcal{T}(w_1) \not\approx_{\mathcal{T}} \mathcal{T}(w_2)$. Overall, we have that there is no classifier for $\mathcal{T}$ that respects $\mathcal{U}$ iff there are words $w, w' \in (2^P)^*$ such that $w \sim_{\mathcal{U}} w'$ and $\mathcal{T}(w) \not\approx_{\mathcal{T}} \mathcal{T}(w')$.

C.3 Missing details in the proof of Theorem 8

► **Lemma 18.** *Every classifier $\mathcal{C}_n$ for $\mathcal{T}_n$ that respects $\mathcal{U}_n$ has at least 2^n states.*

Proof. Let $\mathcal{C}_n = \langle 2^{P_n}, T, t_0, \eta, \kappa \rangle$ be a classifier for $\mathcal{T}_n$ that respects $\mathcal{U}_n$. Consider a state $t \in T$ that is reachable by a word w of even length. Since $\mathcal{C}_n$ respects $\mathcal{U}_n$, and $\mathcal{U}_n$ disables all d -signals in odd positions, it follows that t does not sense any d -signal. I.e., $\text{sensed}(t) \subseteq \{s_0, \dots, s_{n-1}\}$. Recall that letters $v \in 2^{P_n}$ in odd positions in an input word to $\mathcal{T}_n$ are interpreted as pairs $\text{odd}(v) = \langle i, j \rangle \in [n] \times \{0, 1\}$, where i is the number of true d -signals in v and $j = v(s_i)$.

For every integer $i \in [n]$, consider the mapping $\iota_i : 2^{\{s_0, \dots, s_{n-1}\}} \rightarrow T$ defined by $\iota_i(u) = \eta(t, u \cup \{d_{i'} : 1 \leq i' \leq i\})$. Note that since t does not sense any d -signals, it holds that $\iota_i = \iota_{i'}$ for every $i, i' \in [n]$. We thus denote by $\iota : 2^{\{s_0, \dots, s_{n-1}\}} \rightarrow T$ the mapping defined by $\iota(u) = \iota_i(u)$ for some (equivalently, all) $i \in [n]$.

We argue that ι is injective and so $|T| \geq 2^n$. Assume by way of contradiction that there are two different assignments $u^1, u^2 \subseteq \{s_0, \dots, s_{n-1}\}$ such that $\iota(u^1) = \iota(u^2)$. Note that since u^1 and u^2 are different, there is some s_i such that $u^1(s_i) \neq u^2(s_i)$. Let $v_1^1 = u^1 \cup \{d_{i'} : 1 \leq i' \leq i\}$ and



858 $v_1^2 = u^2 \cup \{d_{i'} : 1 \leq i' \leq i\}$. It is not hard to see that $\text{odd}(v_1^1) = \langle i, u^1(s_i) \rangle$ and $\text{odd}(v_1^2) = \langle i, u^2(s_i) \rangle$.
 859 Since $u^1(s_i) \neq u^2(s_i)$, we have that $\text{odd}(v_1^1) \approx \text{odd}(v_1^2)$. Let $v_2 \in 2^{P_n}$ be an assignment such that
 860 $\text{even}(v_2) = \text{odd}(v_1^1)$. Consider the words $w_1 = w \cdot v_1^1 \cdot v_2$ and $w_2 = w \cdot v_1^2 \cdot v_2$. The words w_1 and w_2
 861 are of even length, and $\text{odd}(v_1^1) = \text{even}(v_2)$ and $\text{odd}(v_1^2) \approx \text{even}(v_2)$. Hence, $w_1 \in L_1^n$ and $w_2 \in L_2^n$.
 862 On the other hand, since $\iota = \iota_i$ we have,

$$863 \quad \eta(t_0, w_1) = \eta(t, v_1^1 \cdot v_2) = \eta(\eta(t, v_1^1), v_2) = \eta(\iota_i(u^1), v_2) = \eta(\iota(u^1), v_2).$$

864 Similarly,

$$865 \quad \eta(t_0, w_2) = \eta(t, v_1^2 \cdot v_2) = \eta(\eta(t, v_1^2), v_2) = \eta(\iota_i(u^2), v_2) = \eta(\iota(u^2), v_2).$$

866 Since $\iota(u^1) = \iota(u^2)$, we have that $\eta(t_0, w_1) = \eta(t_0, w_2)$. It follows that $\mathcal{C}_n$ does not distinguish
 867 between w_1 and w_2 , a contradiction. $\blacktriangleleft$

868 **► Lemma 19.** *There exists a classifier $\mathcal{C}_n^*$ for $\mathcal{T}_n$ that respects $\mathcal{U}_n$ with $2^n + 2$ states.*

869 **Proof.** We define $\mathcal{C}_n^* = \langle 2^{P_n}, Q'_n, q'_0, \eta_n, \kappa_n \rangle$ to be the classifier for $\mathcal{T}_n$ that senses only s -signals
 870 in odd positions and senses all signals in even positions, and uses the subset construction on $\mathcal{T}_n$
 871 with respect to the non-determinism introduced by not sensing d -signals in odd positions. As we
 872 shall argue below, all the reachable states in such a subset construction are *legitimate*, in the sense
 873 that they do not include both q_1 and q_2 , and so allow to correctly separate between L_1^n and L_2^n .
 874 Formally, we define the components of $\mathcal{C}_n^*$ as follows.

- 875 ■ For a set $S \in 2^{[n] \times \{0,1\}}$, we say that S is *neat* if S includes exactly one element from each couple.
 876 That is, for every $i \in [n]$, we have that $|S \cap \{\langle i, 0 \rangle, \langle i, 1 \rangle\}| = 1$.
 877 We define $Q'_n = \{\{q_0, q_1\}, \{q_0, q_2\}\} \cup \{S \in 2^{[n] \times \{0,1\}} : S \text{ is neat}\}$. As we shall argue below, when
 878 sensing only s -signals in odd positions, and sensing all signals in even positions, the reachable
 879 states in the subset construction on $\mathcal{T}_n$ are exactly the states in Q'_n .
- 880 ■ $q'_0 = \{q_0, q_1\}$.
- 881 ■ In a state $q' \in \{\{q_0, q_1\}, \{q_0, q_2\}\}$, the classifier uses the odd interpretation of valuations, and
 882 senses only the s -signals. Recall that the d -signals in an assignment $v \in 2^{P_n}$ encode the integer
 883 $i \in [n]$ for which $\text{odd}(v) = \langle i, j \rangle$. Accordingly, the transition function is defined by:

$$884 \quad \begin{aligned} \eta_n(q', v) &= \bigcup_{q \in q'} \bigcup_{i' \in [n]} \{\delta_n(q', \langle i', v(s_{i'}) \rangle)\} \\ 885 \quad &= \bigcup_{i' \in [n]} \{\delta_n(q_0, \langle i', v(s_{i'}) \rangle)\} \\ 886 \quad &= \{\langle i', v(s_{i'}) \rangle : i' \in [n]\}. \end{aligned}$$

888 Note that the equality above is independent of the specific $q' \in \{\{q_0, q_1\}, \{q_0, q_2\}\}$ since δ_n is
 889 defined similarly for all even states. Also, every $i' \in [n]$ contributes to the union exactly one
 890 state among $\langle i', 0 \rangle$ and $\langle i', 1 \rangle$, and so the obtained set is neat.

891 By definition of neat sets, for every neat $S \in 2^{[n] \times \{0,1\}}$ and every letter $v \in 2^{P_n}$, it holds
 892 that either $\text{even}(v) \in S$ or $\langle i, j \rangle \in S$ for the unique $\langle i, j \rangle \approx \text{even}(v)$, but not both. Hence,
 893 the transition function from neat states is defined as follows for every $v \in 2^{P_n}$. It follows
 894 that for every neat $S \in 2^{[n] \times \{0,1\}}$ and every letter $v \in 2^{P_n}$, it holds that $\delta_n(S, v) = \{q_0, q_1\}$ if
 895 $\text{even}(v) \in S$, and $\delta_n(S, v) = \{q_0, q_2\}$ if otherwise $\text{even}(v) \notin S$.

896 We summarize the above. For every $q' \in Q'_n$ and letter $v \in 2^{P_n}$, we have that:

$$897 \quad \eta_n(q', v) = \begin{cases} \{\langle i, v(s_i) \rangle : i \in [n]\} & \text{if } q' \in \{\{q_0, q_1\}, \{q_0, q_2\}\} \\ \{q_0, q_1\} & \text{if } \text{even}(v) \in q' \\ \{q_0, q_2\} & \text{if } \text{even}(v) \notin q'. \end{cases}$$



898 Note that transitions in $\mathcal{C}_n^*$ alternate between states in $\{\{q_0, q_1\}, \{q_0, q_2\}\}$ and neat states in
 899 $2^{[n] \times \{0,1\}}$.

900 ■ For every $S \in Q'_n$, we define $\kappa_n(S) = 1$ if $S = \{q_0, q_1\}$, $\kappa_n(S) = 2$ if $S = \{q_0, q_2\}$, and $\kappa_n(S) = 0$
 901 otherwise.

902 Clearly, by definition of $\mathcal{C}_n^*$, the transition function δ'_n is defined by the subset construction on $\mathcal{T}_n$
 903 with non-determinism introduced due to not sensing the d -signals in odd positions. More formally,
 904 it follows by induction that for every word $w \in (2^{P_n})^*$, we have that $\delta_n(q_0, w) \in \delta'_n(q'_0, w)$. Since
 905 no state in Q'_n includes both q_1 and q_2 , it follows that for every word $w \in (2^{P_n})^*$, if $\delta_n(q_0, w) = q_1$,
 906 then $\delta'_n(q'_0, w) = \{q_0, q_1\}$, and if $\delta_n(q_0, w) = q_2$, then $\delta'_n(q'_0, w) = \{q_0, q_2\}$. Hence, $\mathcal{C}_n^*$ is a correct
 907 classifier for $\mathcal{T}_n$, and since $\mathcal{C}_n^*$ does not sense any d -signal in odd positions, it respects $\mathcal{U}_n$. ◀

908 C.4 Missing details in the proof of Theorem 13

909 We formally define $\mathcal{T}$ and $\mathcal{H}$ as follows. Recall that $L_1 = \text{True}^* \cdot p$ and $L_2 = \text{True}^* \cdot \bar{p}$,
 910 and that H_1 and H_2 are the secrets defined by $H_1 = \{w \in \text{True}^* : \text{run}(w) \text{ is accepting}\}$ and
 911 $H_2 = \{w \in \text{True}^* : \text{run}(w) \text{ is rejecting}\}$, where $\text{run}(w)$ is the run of $\mathcal{N}$ on $\text{word}(w)$ and the non-
 912 determinism of $\mathcal{N}$ is resolved by the assignments to b in w . We show that the specification and
 913 secrets can be defined by transducers of size polynomial in the size of $\mathcal{N}$.

914 The specification $\mathcal{T} = \langle 2^P, \{0, 1, 2\}, \{s_0, s_p, s_{\bar{p}}\}, s_0, \delta_{\mathcal{T}}, \tau_{\mathcal{T}} \rangle$ is defined for every $s \in \{s_0, s_p, s_{\bar{p}}\}$
 915 and $\sigma \in 2^P$, by $\delta_{\mathcal{T}}(s, \sigma) = s_p$ if $\sigma(p) = \text{True}$, and $\delta_{\mathcal{T}}(s, \sigma) = s_{\bar{p}}$ otherwise. The labeling function
 916 $\tau_{\mathcal{T}}$ is defined by $\tau_{\mathcal{T}}(s_0) = 0$, $\tau_{\mathcal{T}}(s_p) = 1$, and $\tau_{\mathcal{T}}(s_{\bar{p}}) = 2$. It is not hard to see that $\mathcal{T}$ defines the
 917 languages $L_1 = \text{True}^* \cdot p$ and $L_2 = \text{True}^* \cdot \bar{p}$ and that the value of b is irrelevant for $\mathcal{T}$.

918 The secret $\mathcal{H} = \langle 2^{p,b}, \{0, 1, 2\}, Q, q_0, \delta, \tau \rangle$ is defined directly on top of $\mathcal{N}$ by adding the labeling
 919 function τ that labels every accepting state $q \in F$ by $\tau(q) = 1$, and every rejecting state $q \in Q \setminus F$
 920 by $\tau(q) = 2$. It is not hard to see that $\mathcal{H}$ defines the secrets H_1 and H_2 as described above.

921 Clearly, $\mathcal{T}$ and $\mathcal{H}$ can be constructed in polynomial time from $\mathcal{N}$, and so the reduction is
 922 polynomial.

923 C.5 Missing details in the proof of Theorem 14

924 The transducer $\mathcal{T}_n$ and the classifier $\mathcal{C}_n$ appear in Figure 3 below.

925 Note that $\mathcal{C}_n$ has $2n$ states and that for every word $w \in \{p, \bar{p}\}^*$, we have that $\mathcal{C}_n(w) = 1$ iff
 926 $w \in (\text{True}^n)^* \cdot p \cdot \text{True}^{n-1}$. Thus, all words in L_1^n are mapped to 1, and all words in L_2^n are mapped
 927 to 2. Also, the limit distribution of the states in $\mathcal{C}_n$ is uniform, and only two states need to sense p .
 928 Hence, $\text{sense}(\mathcal{C}_n) = \frac{1}{n}$, and we are done.

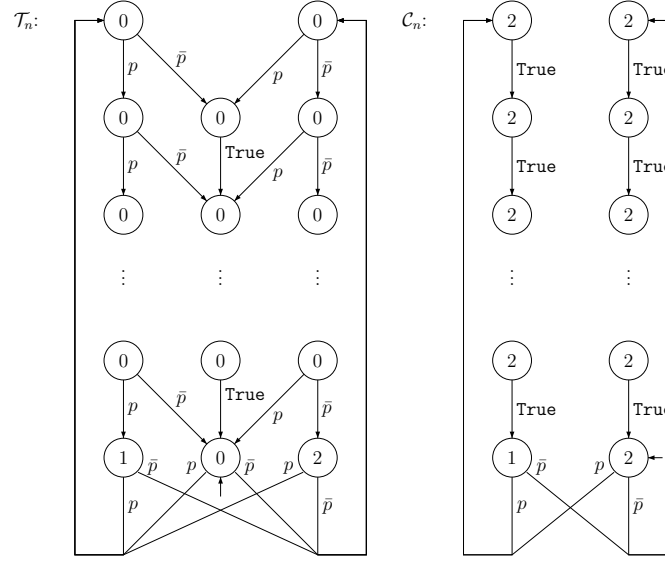
929 We prove that that $\text{sense}(\mathcal{C}) = 1$ for every two-state classifier for $\mathcal{T}_n$. Let $\mathcal{C} = \langle \{p, \bar{p}\}, \{s_1, s_2\}, s, \delta, \tau \rangle$
 930 be classifier with $\text{sense}(\mathcal{C}) < 1$. Let w_1 and w_2 be nonempty words with which reaches s_1 and s_2 ,
 931 respectively. Note that $\text{sense}(\mathcal{C}) < 1$ implies that p is not sensed in at least one of the states, thus
 932 $\text{sense}(s_1) = 0$ or $\text{sense}(s_2) = 0$. Without loss of generality, assume that $\text{sense}(s_1) = 0$.

933 If $\delta(s_1, p) = \delta(s_1, \bar{p}) = s_1$, then either $\tau(s_1) = 1$, in which case $(w_1 \cdot \bar{p}^n) = (w_1 \cdot \bar{p} \cdot \bar{p}^n) =$
 934 $\dots = (w_1 \cdot \bar{p}^{n-1} \cdot \bar{p}^n) = 1$, but at least one these words is in L_2^n , or $\tau(s_1) = 2$, in which case
 935 $(w_1 \cdot \bar{p}^n) = (w_1 \cdot \bar{p} \cdot \bar{p}^n) = \dots = (w_1 \cdot \bar{p}^{n-1} \cdot \bar{p}^n) = 2$, one of which is in L_1^n .

936 If $\delta(s_1, p) = \delta(s_1, \bar{p}) = s_2$, we proceed as follows. First, if $\delta(s_2, p) = \delta(s_2, \bar{p})$, then does not
 937 distinguish between the words p^n and $\bar{p}^n$, and so it clearly does not satisfy $\mathcal{T}_n$.

938 Now, if $\delta(s_2, p) \neq \delta(s_2, \bar{p})$, assume first that $\delta(s_2, p) = s_1$ and $\delta(s_2, \bar{p}) = s_2$. If $\tau(s_2) = 1$, then
 939 $(w_2 \cdot \bar{p}^n) = (w_2 \cdot \bar{p} \cdot \bar{p}^n) = \dots = (w_2 \cdot \bar{p}^{n-1} \cdot \bar{p}^n) = 1$, one of which is in L_2^n . If $\tau(s_2) = 2$ and n is
 940 even, we observe that $(w_2 \cdot p^n) = (w_2 \cdot \bar{p} \cdot p^n) = \dots = (w_2 \cdot \bar{p}^{n-1} \cdot p^n) = 2$, one of which is in L_1^n , and
 941 if n is odd, we observe that $(w_2 \cdot p^{n+1}) = (w_2 \cdot \bar{p} \cdot p^{n+1}) = \dots = (w_2 \cdot \bar{p}^{n-1} \cdot p^{n+1}) = 2$, one of which
 942 is in L_1^n . Finally, if $\delta(s_2, p) = s_2$ and $\delta(s_2, \bar{p}) = s_1$, we again distinguish between the cases $\tau(s_2) = 1$
 943 and $\tau(s_2) = 2$, both lead to wrongly classified words. Thus, does not satisfy $\mathcal{T}$, and we are done.





■ **Figure 3** The specification transducer $\mathcal{T}_n$, and the classifier $\mathcal{C}_n$

C.6 Missing details in the proof of Theorem 15

We start by describing a classifier for $\mathcal{T}_n$ that, as we argue later, has minimal sensing cost.

► **Lemma 20.** *The classifier $\mathcal{C}_n^*$ for $\mathcal{T}_n$ defined in the proof of Lemma 19 has sensing cost $\frac{3n-1}{2}$.*

Proof. By definition of the sensing constraint $\mathcal{U}_n$, in every run of $\mathcal{C}_n^*$, the classifier senses only s -signals in odd positions, and senses all signals in even positions. The set P_n includes $2n - 1$ signals, n of which are s -signals and $n - 1$ of which are d -signals. Hence, on average $\mathcal{C}_n^*$ senses $\frac{n+2n-1}{2} = \frac{3n-1}{2}$ signals per letter, and so $\text{sense}(\mathcal{C}_n^*) = \frac{3n-1}{2}$. ◀

We now argue that the sensing cost of $\mathcal{C}_n^*$ is optimal.

► **Lemma 21.** *Every classifier $\mathcal{S}_n$ for $\mathcal{T}_n$ has $\text{sense}(\mathcal{S}_n) \geq \frac{3n-1}{2}$.*

Proof. Let $\mathcal{C}_n = \langle 2^{P_n}, T, t_0, \rho, \beta \rangle$ be a classifier for $\mathcal{T}_n$. We prove following two claims. Consider a state $t \in T$.

■ If there is a word w of even length such that $\rho(t_0, w) = t$, then $\{s_0, \dots, s_{n-1}\} \subseteq \text{sensed}(t)$.

■ If there is a word w of odd length such that $\rho(t_0, w) = t$, then $\text{sensed}(t) = P_n$.

By the two claims, in each run of the classifier $\mathcal{C}_n$, must sense all $2n - 1$ signals in at least half of the positions, and has to sense n signal in the other positions. Hence $\text{sense}(\mathcal{C}_n) \geq \frac{3n-1}{2}$.

In order to prove the first claim, consider a state $t \in T$ such that there is a word w of even length such that $\rho(t_0, w) = t$. Assume by way of contradiction that $\{s_0, \dots, s_{n-1}\} \not\subseteq \text{sensed}(t)$. Let s_i be a signal that is not sensed in t and let $v_1^1 = \{d_0, \dots, d_{i-1}, s_i\}$ and $v_1^2 = \{d_0, \dots, d_{i-1}\}$ be two assignments that differ only on s_i and with i many true d -signals. Accordingly, $\text{odd}(v_1^1) = \langle i, 1 \rangle$ and $\text{odd}(v_1^2) = \langle i, 0 \rangle$. Since s_i is not sensed in t , we have that $\rho(t, v_1^1) = \rho(t, v_1^2) = t'$. Let $v_2 = \{d_1, s_1, d_2, s_2, \dots, d_i, s_i\}$ be an assignment that includes i many d -signals and i many s -signals. Note that $\text{even}(v_2) = \langle i, 0 \rangle$. Consider the words $w_1 = w \cdot v_1^1 \cdot v_2$ and $w_2 = w \cdot v_1^2 \cdot v_2$. Since $\text{odd}(v_1^1) = \text{even}(v_2)$ and $\text{odd}(v_1^2) \approx \text{even}(v_2)$, it follows that $w_1 \in L_1^n$ and $w_2 \in L_2^n$. On the other hand, $\rho(t_0, w_1) = \rho(t', v_2) = \rho(t_0, w_2)$, and so $\mathcal{C}_n$ does not distinguish between w_1 and w_2 .

We continue to the second claim. Observe first that for every $i \in [n]$ and signal $p \in P_n$, there is a valuation $v \in 2^{P_n}$ that assigns true to exactly $2i$ signals excluding p , and there is a valuation

970 $v' \in 2^{P_n}$ that differs from v only on p and thus assigns true to exactly $2i + 1$ signals including p .
 971 Such valuations have the even interpretations $\langle i, 0 \rangle$ and $\langle i, 1 \rangle$ respectively, and so are a couple.

Consider a state $t \in T$ such that there is a word y of even length and an assignment $v_1 \in 2^{P_n}$ such that $\rho(t_0, y \cdot v_1) = t$. Assume by way of contradiction that there is some signal $p \in P_n$ not in $\text{sensed}(t)$. Let $\text{odd}(v_1) = \langle i, j \rangle$. By the observation above, there are two valuations v_2^1 and v_2^2 that differ only in the value of p and with $\text{even}(v_2^1) = \text{odd}(v_1)$ and $\text{even}(v_2^2) \approx \text{odd}(v_1)$. Consider the words $w_1 = y \cdot v_1 \cdot v_2^1$ and $w_2 = y \cdot v_1 \cdot v_2^2$. Since w_1 and w_2 are two even-length words and by choice of v_2^1 and v_2^2 , we have that $w_1 \in L_1^n$ and $w_2 \in L_2^n$. On the other hand, since p is not sensed in t and v_2^1 and v_2^2 differ only in their assignment to p , we have that

$$\rho(t_0, w_1) = \rho(t, v_2^1) = \rho(t, v_2^2) = \rho(t_0, w_2).$$

972 As in the previous case, it is not hard to see that $\mathcal{C}_n$ does not distinguish between the words w_1
 973 and w_2 . ◀

974 Finally, we argue that a classifier for $\mathcal{T}_n$ that attains the optimal sensing cost must have
 975 exponentially many states.

976 ▶ **Lemma 22.** *Every classifier $\mathcal{C}_n$ for $\mathcal{T}_n$ such that $\text{sense}(\mathcal{C}_n) = \frac{3n-1}{2}$ has at least 2^n states.*

977 **Proof.** Let $\mathcal{C}_n = \langle 2^{P_n}, T, t_0, \rho, \beta \rangle$ be a classifier for $\mathcal{T}_n$. By Lemma 21, all the states in T that
 978 are reachable by a word of an odd length must sense all signals and all other states must sense
 979 all s -signals. Having $\text{sense}(\mathcal{C}_n) = \frac{3n-1}{2}$ is possible only when $\mathcal{C}_n$ eventually alternates between
 980 states that are reachable only by words of odd length and states that are reachable only by words
 981 of even length, with the latter sensing only s -signals. Thus, in the limit, every classifier $\mathcal{C}_n$ with
 982 $\text{sense}(\mathcal{C}_n) = \frac{3n-1}{2}$ behaves as a classifier that respects $\mathcal{U}_n$ as described in Theorem 8. In other
 983 words, if $\mathcal{C}_n$ has $\text{sense}(\mathcal{C}_n) = \frac{3n-1}{2}$, then there is a reachable even state t of $\mathcal{C}_n$, such that $\mathcal{C}_n$ when
 984 initialized at t is a classifier for $\mathcal{T}_n$ that respects $\mathcal{U}_n$. By Lemma 18, it follows that $\mathcal{C}_n$ has at least
 985 2^n states, and we are done. ◀

986 C.7 Missing details in the proof of Theorem 17

987 Let $\mathcal{M}_{\mathcal{T}} = \langle \text{legit}(\mathcal{T}), \text{legit}(\mathcal{T}) \times 2^P, \{\langle q_0, 0 \rangle\}, P, \text{cost} \rangle$. We prove that $\mathcal{N}$ is universal if and only if
 988 $\text{sense}(\mathcal{M}_{\mathcal{T}}) \geq |P_{\text{base}}| + 1$. Let $v_{\perp}$ denote the failure sink state that any valid sensing policy must
 989 avoid with probability 1.

990 C.7.1 Mandatory sensing constraints.

991 We first identify signals that a valid sensing policy g must necessarily sense for every reachable
 992 state $S \in \text{legit}(\mathcal{T})$.

- 993 ■ **Sensing P_{base} :** For every signal $p \in P_{\text{base}}$, there exists a reset signal $r \in \{r_1, r_2\}$ such that
 994 $r \neq p$. The assignments $a_1 = \{r\}$ and $a_2 = \{r, p\}$ are both reset assignments but yield opposite
 995 parities over P_{base} . Thus, $\tau(\delta(\langle q, c \rangle, a_1)) = 1$ and $\tau(\delta(\langle q, c \rangle, a_2)) = 2$ for all $\langle q, c \rangle \in S$. If
 996 $p \notin g(S)$, g cannot distinguish between a_1 and a_2 , causing a transition to $v_{\perp}$ with positive
 997 probability. Therefore, $P_{\text{base}} \subseteq g(S)$ for all S .
- 998 ■ **Sensing s :** If S intersects $\alpha \times \{0, 1, 2\}$, there exists $\langle q, c \rangle \in S$ with $q \in \alpha$. The assignments
 999 $a_1 = \{s\}$ and $a_2 = \emptyset$ lead to $\delta'(\langle q, c \rangle, a_1) = \langle q', 1 \rangle$ and $\delta'(\langle q, c \rangle, a_2) = \langle q', 2 \rangle$, respectively, where
 1000 $q' = \delta(q, \emptyset)$. If $s \notin g(S)$, g cannot distinguish a_1 from a_2 , leading to $v_{\perp}$. Therefore, $s \in g(S)$
 1001 whenever $S \cap (\alpha \times \{0, 1, 2\}) \neq \emptyset$.



C.7.2 Safety of masking b .

Unlike P_{base} and s , sensing b is never strictly required to maintain safety. Consider a state $S \in legit(\mathcal{T})$ where $b \notin g(S)$:

- **Under simulation assignments:** For any $a, a' \in 2^P$ such that $a \sim_{P_{base}} a'$ but $a(b) \neq a'(b)$, the target states $\delta'(\langle q, c \rangle, a) = \langle p, j \rangle$ and $\delta'(\langle q, c \rangle, a') = \langle p', j' \rangle$ satisfy $\{j, j'\} \subseteq \{0, i\}$, where $i \in \{1, 2\}$ is the parity of P_{base} in a . The next position $S' = \delta'(S, a)$ will strictly satisfy $\tau(S') \subseteq \{0, i\}$, guaranteeing $S' \in legit(\mathcal{T})$.
- **Under reset assignments:** The target classification $\delta'(\langle q, c \rangle, a)$ depends strictly on $a(s)$, thus completely independent of b .

Therefore, the policy g^* that continuously senses P_{base} , senses s strictly when $S \cap (\alpha \times \{0, 1, 2\}) \neq \emptyset$, and permanently masks b , is a strictly safe policy.

C.7.3 Cost analysis: The universal case.

Assume $\mathcal{N}$ is universal. We analyze an arbitrary valid policy g and prove $sense(\mathcal{M}_{\mathcal{T}}^g) \geq |P_{base}| + 1$, with equality holding if and only if $g = g^*$.

Because g must sense P_{base} globally, we partition the state space of $\mathcal{M}_{\mathcal{T}}^g$ based on the discretionary signals sensed: $X_{\emptyset}$, $X_{\{b\}}$, $X_{\{s\}}$, and $X_{\{b,s\}}$, where for every $\gamma \subseteq \{b, s\}$, X_{γ} denotes the set of states where exactly the signals in γ are sensed. Let $\pi(X_{\gamma})$ be the limiting probability of being in X_{γ} . Because $g(S) = P_{base} \cup \gamma$ for every $S \in X_{\gamma}$, the expected limit-average cost is

$$sense(\mathcal{M}_{\mathcal{T}}^g) = |P_{base}| + 2\pi(X_{\{b,s\}}) + \pi(X_{\{b\}}) + \pi(X_{\{s\}}).$$

Using $\sum \pi(X_{\gamma}) = 1$, the inequality $sense(\mathcal{M}_{\mathcal{T}}^g) \geq |P_{base}| + 1$ reduces to proving $\pi(X_{\{b,s\}}) \geq \pi(X_{\emptyset})$.

Because the transition function of $\mathcal{T}$ depends only on the Q -component and the assignment, any two legitimate states S, S' with identical Q -projections share identical transition probabilities to subsequent Q -projections. We may safely assume an optimal memoryless policy depends solely on the Q -projection. Thus, $\mathcal{M}_{\mathcal{T}}^g$ acts as a Markov Chain over $2^Q \cup \{v_{\perp}\}$. This guarantees a unique, single baseline state $S^* = \{q_0\}$ reached deterministically upon any reset assignment.

An excursion from S^* is a random walk $S_0, S_1, \dots$ in $\mathcal{M}_{\mathcal{T}}^g$ that starts at S^* and ends upon returning to S^* . Since S^* is recurrent, every excursion eventually returns to S^* with probability 1. Let $\ell > 0$ be a random excursion length. Because a reset transition forces a return to S^* , and simulation transitions occur with probability $\frac{1}{4}$, it follows that $\mathbf{E}[\ell] \leq \frac{4}{3}$. For a subset X , let $\#X$ denote the number of visits to X during a single excursion. We utilize the Renewal Reward Theorem to evaluate π via excursions from the recurrent state S^* as follows. The limiting probability is $\pi(X) = \frac{\mathbf{E}[\#X]}{\mathbf{E}[\ell]}$. Thus, proving $\pi(X_{\{b,s\}}) \geq \pi(X_{\emptyset})$ is equivalent to proving $\mathbf{E}[\#X_{\{b,s\}}] \geq \mathbf{E}[\#X_{\emptyset}]$.

During an excursion from S^* , s must be sensed continuously until and including the first time b is sensed. Indeed, if otherwise $S_i \in X_{\emptyset}$ for $0 \leq i < \ell$ such that $S_j \in X_{\{s\}}$ for all $j < i$, then the current state S_i corresponds precisely to the subset construction $\delta(q_0, \text{word}(w^i))$, where w^i is the prefix read up to position i . Because $\mathcal{N}$ is universal, $S_i \cap \alpha \neq \emptyset$. Failing to sense s at this juncture would cause an immediate transition to $v_{\perp}$ upon a reset transition. Consequently, all visits to $X_{\emptyset}$ must strictly follow the first visit to $X_{\{b,s\}}$ if it exists.

Let ℓ^{post} be the number of transitions following the first round b is sensed. It follows that $\#X_{\emptyset} \leq \ell^{post}$. Since $\ell^{post} = 0$ whenever $\#X_{\{b,s\}} = 0$, we have

$$\mathbf{E}[\ell^{post}] = \mathbf{E}[\ell^{post} \mid \#X_{\{b,s\}} \geq 1] \cdot \Pr(\#X_{\{b,s\}} \geq 1).$$

Given b is sensed, the expected number of remaining simulation transitions before a reset is exactly $\frac{1/4}{1-1/4} = \frac{1}{3}$. All ℓ^{post} remaining transitions are simulation transitions, as the excursion ends immediately upon a reset transition. Therefore, $\mathbf{E}[\ell^{post} \mid \#X_{\{b,s\}} \geq 1] \leq \frac{1}{3}$. Because



1048 $\Pr(\#X_{\{b,s\}} \geq 1) \leq \mathbf{E}[\#X_{\{b,s\}}]$, we obtain

$$1049 \quad \mathbf{E}[\#X_\emptyset] \leq \mathbf{E}[\ell^{post}] \leq \frac{1}{3} \mathbf{E}[\#X_{\{b,s\}}] \leq \mathbf{E}[\#X_{\{b,s\}}],$$

1050

1051 with equality holding if and only if $\#X_\emptyset = \#X_{\{b,s\}} = \#X_{\{b\}} = 0$ with probability 1, yielding
 1052 $\pi(X_{\{s\}}) = 1$ and exactly $\text{sense}(\mathcal{M}_\mathcal{T}) = |P_{base}| + 1$.

1053 **C.7.4 Cost analysis: the non-universal case.**

1054 Assume $\mathcal{N}$ is not universal. We prove $\text{sense}(\mathcal{M}_\mathcal{T}) < |P_{base}| + 1$ by analyzing the performance of
 1055 policy g^* .

1056 Because $\mathcal{N}$ is not universal, there exists a word $x \in \Sigma^*$ such that $\delta(q_0, x) \cap \alpha = \emptyset$. When
 1057 Nature generates $w \in (2^P)^*$ such that $\text{word}(w) = x$ with no intervening resets, the MDP reaches
 1058 the state $S = \delta(q_0, x)$. Because $S \cap \alpha = \emptyset$, the policy g^* safely assigns $s \notin g^*(S)$, placing $S \in X_\emptyset$.
 1059 Because the sequence corresponding to x occurs with strictly positive probability $\frac{1}{|\Sigma|^{|x|}}$ immediately
 1060 following any reset assignment, the state S belongs to the same ergodic component as S^* and is
 1061 positive recurrent. Consequently, $\pi(X_\emptyset) > 0$. Because g^* never senses b , $\pi(X_{\{b,s\}}) = \pi(X_{\{b\}}) = 0$.
 1062 The limit-average cost evaluates strictly to

$$1063 \quad (|P_{base}| + 1)\pi(X_{\{s\}}) + |P_{base}|\pi(X_\emptyset) = (|P_{base}| + 1) - \pi(X_\emptyset) < |P_{base}| + 1.$$

1064

1065 Hence, $\text{sense}(\mathcal{M}_\mathcal{T}) < |P_{base}| + 1$.